UNIVERSITY OF ST. GALLEN

School of Management, Economics,
Law, Social Sciences, International Affairs and Computer Science

Report

CryptoFlow

Event-Driven and Process-Oriented Architectures

Submitted by:
Ioannis Theodosiadis - 25-603-457
Cyril Gabriele - 21-558-358

Approved on Application by:
Prof. Dr. Barbara Weber
Dr. Amine Abbad-Andaloussi

Date of Submission:
22.05.2026

Abstract

CryptoFlow is an event-driven crypto portfolio simulation platform built with Apache Kafka, Camunda 8, Kafka Streams, and Spring Boot. Developed for the Event-Driven and Process-Oriented Architectures (EDPO) course at the University of St. Gallen, the system demonstrates how loosely coupled microservices can collaborate through asynchronous event streams, continuously running stream-processing topologies, and orchestrated business processes. The platform ingests real-time cryptocurrency prices and order-book snapshots from Binance, maintains portfolio valuations through both Event-Carried State Transfer and Kafka Streams state stores, processes simulated trading orders through a BPMN-orchestrated workflow backed by bid/ask stream matching, and handles user onboarding as a parallel saga with compensation logic. This report documents the system architecture, the project's architectural decision records, Kafka reliability experiments, stream-processing extensions, and lessons learned throughout the project.

The Project release for this version of the report can be found on GitHub:

https://github.com/cyrilgabriele/EDPO-Project-FS26/releases/tag/v1.0.0

Table of Contents

List of Figures

List of Tables

1 Project Description

CryptoFlow is a crypto portfolio simulation platform that demonstrates event-driven, stream-processing, and process-oriented architecture patterns in a distributed microservice environment. The system allows users to register, manage cryptocurrency portfolios, observe live market data, and place simulated trading orders, all coordinated through asynchronous event streams, materialized stream state, and orchestrated business processes.

The project release for this version of the report can be found on GitHub:

https://github.com/cyrilgabriele/EDPO-Project-FS26/releases/tag/v1.0.0

1.1 Domain and Goals

The central goal of this project was to implement the concepts covered in the EDPO lectures in a self-chosen project. For that purpose, CryptoFlow was designed as a high-level cryptocurrency platform in which users can register, observe live market prices, manage simulated portfolios, and place simulated trading orders. The platform is intentionally not a production exchange; it serves as a concrete case study for applying the architectural and integration concepts from the course in one coherent end-to-end system.

1.1.1 Bounded Contexts

Figure 1 shows the domain as a DDD context map. The final platform is best understood as six bounded contexts: Market Data, Reference Data, Portfolio, Trading, User, and Onboarding.

TODO Ioannis: Replace Figure 1 with the most recent context/topology image before hand-in.

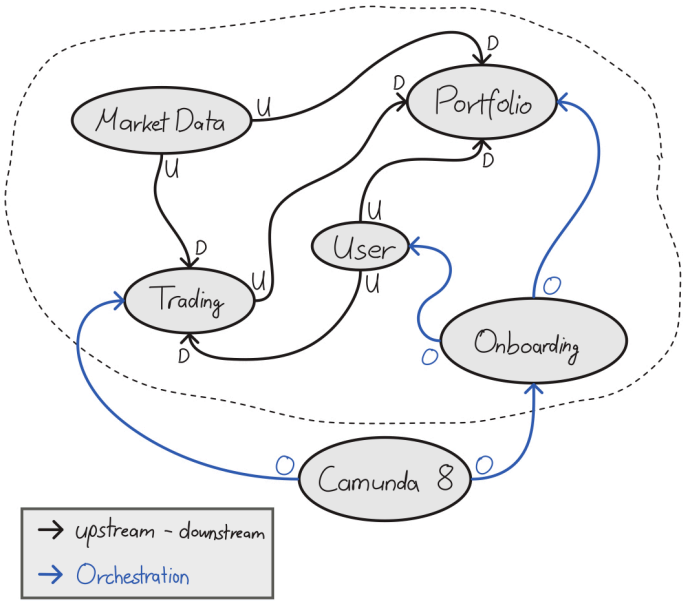


Figure 1: Context map of CryptoFlow showing bounded contexts, upstream/downstream relationships, and external systems

Market Data Upstream supplier for live prices. Ingests external Binance feeds through an Anti-Corruption Layer and publishes price events that flow downstream to Portfolio and Trading.

Reference Data Slow-moving externally sourced facts for event enrichment. Publishes FX rates and coin metadata as compacted reference topics so downstream services can enrich local stream state without synchronous provider calls.

Portfolio Downstream consumer of both price events and approved-order events. Owns holdings, valuation logic, and an in-memory price cache. Participates in the onboarding saga as a Camunda job worker.

Trading Downstream consumer of price events and user-confirmation events. Owns order lifecycle, the transactional outbox, and a replicated read-model for user validation. Deploys the `placeOrder` BPMN process.

User Upstream supplier of confirmed-user events consumed by Trading. Owns user accounts, confirmation links, and identity state. Participates in the onboarding saga as a Camunda job worker.

Onboarding Dedicated saga orchestrator with no persistent data of its own. Coordinates user and portfolio creation across bounded contexts via Camunda 8, keeping orchestration separate from domain ownership.

User and Portfolio are connected through a Partnership relationship: bidirectional compensation events allow each side to roll back the other during onboarding failures. All contexts share a common event schema through the `shared-events` Shared Kernel module, ensuring compile-time consistency.

1.1.2 Architecture Characteristics

The bounded contexts and flows described above impose a set of quality attributes that any architectural solution must satisfy. The worksheet below identifies seven driving characteristics derived from the domain requirements, marks the top three, and lists additional characteristics that were considered but not deemed critical.

Characteristic	Top 3	Rationale
Data integrity	✓	Domain entities transition through distinct lifecycle states and are referenced across bounded contexts. If a state change is recorded in one context but its downstream effect is lost, the contexts diverge from each other. The platform must guarantee that every state change is durably captured and propagated without silent loss.

Fault tolerance	✓	Processes can span multiple bounded contexts that can fail independently after a peer has already succeeded. Some contexts also depend on external systems that may disconnect at any time. The system must handle partial failures, malformed messages, and service crashes without corrupting state or blocking other work.
Availability	✓	Each bounded context must remain functional when its peers are temporarily unreachable. This is the characteristic the platform actively buys by accepting eventual consistency: ECST, replicated read-models, and compensation sagas exist so that Portfolio can value holdings while User is down, Trading can match orders while Portfolio lags, and no single failure cascades across contexts.
Data consistency		Business processes span multiple bounded contexts, yet there is no shared database. Rather than enforcing strict cross-context consistency as an architectural driver, the platform intentionally trades it for availability and converges to correct state eventually through ECST, sagas, and compensation events.
Responsiveness		External market conditions change continuously. The platform must reflect relevant state changes with minimal delay so that downstream consumers can operate on current information.
Deployability		Independently owned bounded contexts must be developable, testable, and releasable without coordinated rollouts. The local development environment must be reproducible and startable with minimal manual steps.
Extensibility		The platform evolves iteratively. New bounded contexts and event flows are added over time. The internal structure of each service must allow adapters and integration technologies to change without rewriting domain logic.

Table 1: Architecture characteristics worksheet

Others considered. Scalability (relevant for a production exchange but not a driving concern for this educational platform), testability (desirable but did not constrain architectural choices), interoperability (only relevant at the Binance boundary), and recoverability (closely related to fault tolerance but not an independent driver).

1.1.3 Implemented Concepts

The following EDPO lecture concepts are implemented in CryptoFlow. They are grouped by the two main parts of the course so the report distinguishes process-oriented integration work from event-driven and stream-processing work.

1.1.3.1 Process-Oriented Concepts

- **Process orchestration** with Camunda 8 / Zeebe, using BPMN workflows to coordinate long-running processes.
- **Service autonomy through bounded contexts** with clear ownership, a database-per-service model, and a dedicated onboarding-service to avoid a process monolith.
- **Parallel Saga** for the onboarding workflow.
- **Fairy Tale Saga** for the `placeOrder` workflow.
- **Compensation mechanisms** for distributed onboarding failures.
- **Transactional Outbox** for reliable publication of approved-order events.
- **Idempotent consumer** handling for at-least-once event delivery in portfolio updates.
- **Replicated read-model** for local validation without synchronous cross-service calls.
- **Human intervention as a stateful resilience pattern** for deterministic workflow failures.

1.1.3.2 Event-Driven and Stream-Processing Concepts

- **Event-driven communication** through Apache Kafka as the backbone for inter-service collaboration.
- **Event-Carried State Transfer (ECST)** for price replication, portfolio updates, compensation events, and replicated user-validation data.
- **Stateless stream processing** for FX-rate ingestion, coin metadata ingestion, and Market Scout ask filtering/translation.
- **Stateful Kafka Streams processing** for bid/ask matching, OHLC windows, Market Scout summaries, and portfolio valuation.
- **Windowing, event-time timestamp extraction, grace periods, and suppression** for closed OHLC bars and market-scout summaries.
- **Joins and repartitioning** through OHLC metadata enrichment, portfolio valuation table-table joins, and portfolio value aggregation by user.
- **Interactive queries** for portfolio value and Market Scout dashboard state.
- **Schema-managed derived events** with Avro and Confluent Schema Registry for cross-service stream contracts.

1.2 System Overview

Figure 2 gives a high-level deployment view of CryptoFlow.

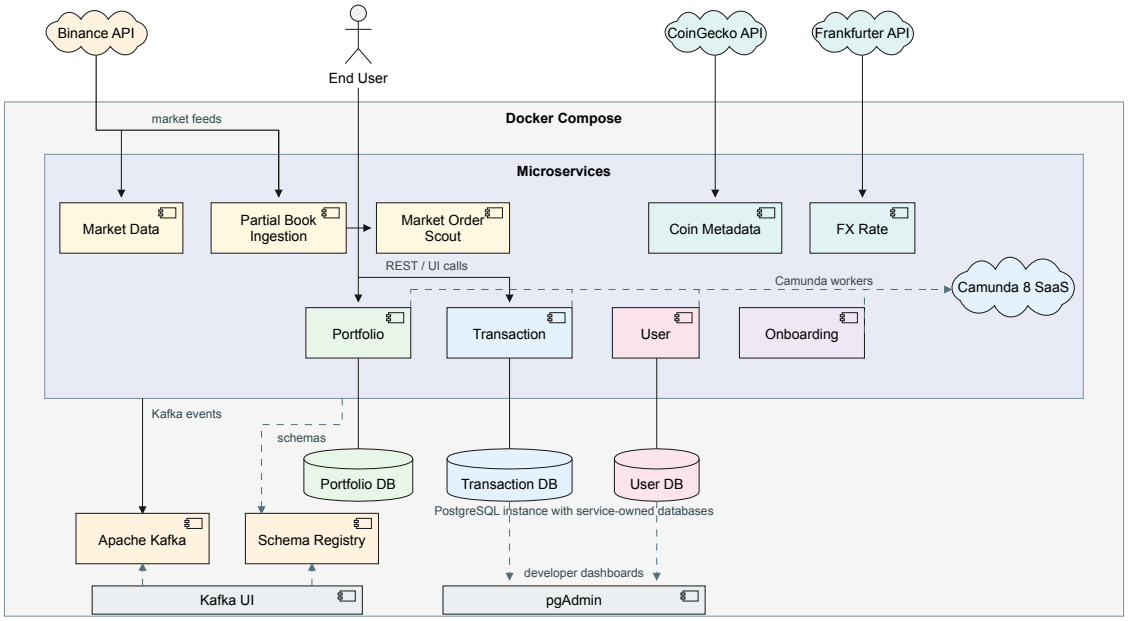


Figure 2: High-level deployment overview showing the Docker Compose runtime, Spring Boot services, infrastructure, and external systems

Inside the Docker Compose boundary, the Spring Boot services communicate through Apache Kafka and persist state in service-owned PostgreSQL databases where needed. Market and reference-data ingestion services are stateless; Portfolio, Transaction, and User each own a dedicated database. Kafka Streams applications materialize local RocksDB-backed state stores for matching, OHLC, Scout dashboard statistics, and portfolio valuation. Kafka UI and pgAdmin provide developer-facing observability.

Outside the local runtime, four external systems integrate with the platform. Binance delivers real-time ticker and partial-book feeds over WebSocket, Frankfurter provides FX rates, CoinGecko provides coin metadata, and Camunda 8 hosts the BPMN process engine with services connecting as stateless gRPC workers. End users interact through REST endpoints exposed by Portfolio and Transaction, and through Camunda Tasklist for human-intervention tasks.

The two main end-to-end flows that define the platform from the outside are the onboarding flow, which creates a confirmed user together with a matching portfolio, and the trading flow, which matches pending buy bids against ask liquidity derived from live order-book snapshots before propagating approved trades to the portfolio context. Chapter 4 provides the service topology, event topology, and BPMN interaction model.

1.3 Technology Stack

Table 2 summarises the technology choices. Where a technology was evaluated through a formal architectural decision, the corresponding ADR is referenced.

Technology	ADR	Rationale
Java 21 + Spring Boot 3.5.11	–	Provides the service runtime, dependency injection, web layer, scheduling, JPA integration, and Kafka/Camunda adapters.
Apache Kafka (Confluent 7.6, KRaft)	Section 6.1	Sole inter-service communication channel for domain events and stream inputs. Topics are explicitly declared via Spring `NewTopic` beans.
Camunda 8 / Zeebe (SaaS)	Section 6.8	Orchestrates multi-step BPMN processes. Services act as stateless gRPC job workers. SaaS deployment provides managed scalability and Operate dashboard.
PostgreSQL 16 + Flyway	Section 6.7, Section 6.19	Stores persistent data for the Portfolio, User, and Transaction bounded contexts with one database per service. Flyway manages schema migrations; Hibernate runs in `validate` mode only.
Docker Compose	–	Provisions Kafka, Schema Registry, PostgreSQL, Kafka UI, pgAdmin, and all application services for reproducible local integration.
Binance WebSocket APIs	Section 6.6, Section 6.21	Provides spot ticker data for prices and USD-M futures partial-book depth for Market Scout without request-time polling.
Frankfurter FX API	Section 6.29	Provides scheduled USD-based FX reference data for display-currency conversion through compacted Kafka topics.
CoinGecko API	Section 6.33	Provides slow-moving coin metadata used to enrich closed OHLC bars through a `GlobalKTable` join.
Kafka Streams	Section 6.22, Section 6.27, Section 6.31, Section 6.34	Runs the continuously active topologies for Market Scout derivation, bid/ask matching, OHLC aggregation, and portfolio valuation.
Confluent Schema Registry + Avro	Section 6.32	Provides schema-managed contracts for cross-service derived events, while raw replay topics stay JSON.
Jackson JSON	Section 6.3	Standardised serialization for raw replay boundaries and original domain events. Readable payloads, but no broker-side schema validation.
`shared-events` module	Section 6.5	Shared Maven module defining Kafka event contracts. Ensures compile-time consistency across all producing and consuming services.

Table 2: Technology stack

2 Team Responsibilities

The project was developed by a two-person team over the course of the FS2026 semester. Both team members contributed to architectural design and system integration, while individual service ownership was divided by bounded context. We kept the contribution balanced between both members.

Table 3 summarizes the concrete ownership split and the major implementation responsibilities of each team member.

Team Member	Area of Ownership	Description
Ioannis	• Architecture design • Place order process • Portfolio Service • Transaction Service • Reference data services • OHLC and portfolio valuation streams • Infrastructure • Kafka experiments • Schema Registry and Avro contracts • Documentation	Designed the overall system architecture and service structure. Owned and implemented the place order flow, transaction-service orchestration, portfolio-service persistence and Kafka consumption, and the second-half portfolio valuation topology with interactive queries. Implemented the Reference Data additions around FX rates, coin metadata, display-currency integration, Schema Registry/Avro contracts, and OHLC enrichment. Led infrastructure setup for local integration, contributed to Kafka reliability experiments, authored architecture decisions, and report documentation.
Cyril	• Architecture design • Onboarding process • Market Data Service • Partial book ingestion • Market Order Scout • Bid/ask matching extension • User Service • Onboarding Service • Kafka experiments • Documentation	Co-designed the system architecture. Owned and implemented the onboarding process end to end across the onboarding service and user service, including the `userOnboarding.bpmn` orchestration, Camunda workers, confirmation handling, compensation behavior, and display-currency preference publication. Built the market data service and the market-scout extension: Binance ticker ingestion, partial-book ingestion, ask-side filtering and Avro-derived scout events, dashboard state, and the bid/ask matching integration with transaction-service. Contributed to Kafka reliability experiments, authored architecture decisions, and report documentation.

Table 3: Overview of team responsibilities and contributions

3 Lecture Concepts

This chapter connects the main EDPO lecture concepts to the CryptoFlow implementation. The first section keeps the process-oriented material from the first half of the course: Kafka as integration backbone, orchestration, service boundaries, sagas, compensation, outbox, idempotency, read models, and human intervention. The second section extends the same mapping to the event-driven and stream-processing concepts from the second half of the course.

3.1 Process-oriented Concepts

3.1.1 Event-Driven Communication through Apache Kafka

Lecture 1 introduced Kafka as the technical backbone for distributed event streams, and Lecture 2 framed event-driven systems as communication through published facts instead of remote commands. CryptoFlow adopts this as its default integration style. Section 6.1 defines Kafka as the sole inter-service communication channel for domain behavior, which means that the services do not call each other synchronously through REST or RPC.

In practice, `market-data-service` publishes `CryptoPriceUpdatedEvent` records to `crypto.price.raw`, Market Scout publishes ask-side liquidity streams for trading, reference-data services publish compacted FX and metadata streams, `user-service` publishes confirmed-user and display-currency events, `transaction-service` publishes approved-order events for portfolio propagation, and the onboarding rollback path uses dedicated compensation topics. Section 6.3 standardizes JSON for the original raw topics, Section 6.32 introduces Avro with Schema Registry for new cross-service derived events, Section 6.4 preserves per-symbol ordering through the Kafka key, and Section 6.5 keeps shared event contracts in the `shared-events` module.

3.1.2 Event-Carried State Transfer

Lecture 2 presented Event-Carried State Transfer (ECST), amongst others, as the pattern in which an event carries enough state for consumers to maintain their own local view instead of querying the source service. CryptoFlow applies ECST in the core domain flows and extends it in the second half with compacted reference-data topics.

First, Section 6.2 replicates market prices from `market-data-service` into the `portfolio-service` price cache, so portfolio valuation reads the locally maintained state instead of calling the producer. Second, Section 6.15 keeps portfolio updates autonomous: after an order is approved, `transaction-service` publishes `OrderApprovedEvent`, and `portfolio-service` updates holdings independently. Third, Section 6.11 uses event-carried compensation requests so user and portfolio data can be deleted across service boundaries without synchronous

rollback calls. Fourth, Section 6.17 uses the same principle for user validation: `transaction-service` maintains a local confirmed-user projection from `UserConfirmedEvent` messages and validates new orders locally. The second-half extension applies the same cache-as-topic idea to FX rates, coin metadata, display currency, and portfolio values through compacted topics.

Display Currency is the clearest second-half ECST example. `user-service` owns the user's ISO-4217 display preference, persists it with a default of `USD`, exposes `PATCH /users/{id}/display-currency`, and publishes `UserDisplayCurrencyUpdated` to the compacted `user.display-currency` topic (Section 6.28). `portfolio-service` and `transaction-service` consume that topic into local caches. Portfolio converts holdings at API read time from its USDT price and FX caches, while Transaction uses the same local display-currency, FX, and price state for the buy-time quote widget and stores the display price shown at order placement. No dashboard or order path needs a synchronous call back to `user-service` or `fx-rate-service`.

3.1.3 Process Orchestration with Camunda 8 with Zeebe

Lectures 3 and 4 introduced workflow engines, BPMN, durable waiting states, and message correlation. CryptoFlow implements these ideas with Camunda 8 / Zeebe, as formalized in Section 6.8.

The alternative would have been Camunda 7 (Operaton), which embeds the engine inside the application and stores process state in the application database. For CryptoFlow this is a poor fit for two reasons. First, `placeOrder` instances wait at an event-based gateway for a price match correlated by `transactionId`. With many concurrent open offers, an embedded engine would require a shared polling table or in-memory workaround, whereas Zeebe's partitioned log lets each instance wait independently without job-lock contention. Second, the onboarding and order-notification workflows use Kafka consumers and SMTP delivery. In Camunda 8, these steps are handled through connector templates directly inside the BPMN model, so the application code contains no notification client or additional Kafka producer for orchestration concerns. With an embedded engine, each integration would need separate application-level client code.

Two BPMN processes are central. `onboarding-service` deploys `userOnboarding.bpmn`, which coordinates user preparation, email confirmation, timeout handling, and the post-confirmation creation steps. `transaction-service` deploys `placeOrder.bpmn`, which coordinates order submission, the wait for `transaction.order-matched`, timeout handling, and the final notification path. In both cases, Zeebe owns the process state, timers, and correlation logic, while the services participate through stateless job workers. This keeps the application databases free of workflow state and allows long-running waits without blocking threads or database

transactions. Camunda Operate provides runtime visibility into all in-flight and completed instances, which is especially useful for debugging stuck or rejected orders.

3.1.4 Service Autonomy through Bounded Contexts

Lecture 4 emphasized that workflow ownership must not collapse service boundaries into a process monolith. CryptoFlow applies this through explicit bounded contexts and a database-per-service model.

The system is split into Market Data, Reference Data, Portfolio, Trading, User Identity, and Onboarding areas, implemented by nine deployable Spring Boot services. Section 6.19 establishes one database per stateful service, while Section 6.7 and Section 6.20 specify the ownership of the portfolio and trading contexts in detail. Cross-service references are opaque identifiers rather than foreign keys or shared tables, and services depend on Kafka events or local projections rather than direct SQL access.

The most important architectural consequence is Section 6.10: onboarding orchestration moved into a dedicated `onboarding-service`. This keeps the cross-context registration flow in one explicit process owner while preserving the autonomy of `user-service` and `portfolio-service`, each of which still owns its own data and local transaction boundary.

3.1.5 Saga Patterns

Lecture 5 introduced multiple transactional saga variants. CryptoFlow implements two different orchestrated saga styles because onboarding and order execution have different coupling and consistency requirements.

3.1.5.1 Parallel Saga

Section 6.10 explains the decisions for the implementation of the onboarding service (`userOnboarding.bpmn`) as a Parallel Saga. After the confirmation message arrives, `userOnboarding.bpmn` reaches a parallel gateway and starts `userCreationWorker` in `user-service` and `portfolioCreationWorker` in `portfolio-service` concurrently. Each worker performs only its own local transaction in its own bounded context. The saga proceeds only if both branches report success. Otherwise the flow moves into compensation for both branches. See Section 3.1.6 for full detail.

This fits the lecture pattern well: communication between participants is asynchronous, consistency is eventual, and coordination remains centralized in one orchestrator. CryptoFlow chose this style because onboarding spans multiple services, includes a e-mail confirmation wait, and must remain visible as one end-to-end process.

3.1.5.2 Fairy Tale Saga

Section 6.13 explains the decisions for the implementation of the place order process (`placeOrder.bpmn`) as a Fairy Tale Saga. The process keeps synchronous service-task interactions inside the trading context, but it accepts eventual consistency at the boundary to the portfolio context. Zeebe can wait at the event-based gateway for either a stream-produced order match or a timeout without holding a distributed lock or an open database transaction.

Once the order is approved in `transaction-service`, that trading decision is terminal in its bounded context. The downstream portfolio update is deliberately not folded into the same atomic step. Instead, the order is approved locally and then propagated asynchronously, which is exactly the trade-off of the Fairy Tale Saga used in the lecture.

3.1.6 Compensation Mechanisms

Compensation is required in CryptoFlow where the business invariant is all-or-nothing: after onboarding, the system should not keep only a user or only a portfolio. Section 6.11 therefore models rollback as explicit forward recovery rather than as a distributed transaction.

If one onboarding branch succeeds and the other fails, the BPMN process routes to compensation handlers that delete the already created entity and publish a compensation request for the peer service. `UserCompensationRequestedEvent` and `PortfolioCompensationRequestedEvent` carry the data needed for the remote delete, and the receiving services treat the delete operation idempotently so at-least-once delivery remains safe. Section 6.12 complements this design: `userCreationWorker` and `portfolioCreationWorker` always complete their Zeebe jobs and report the outcome via `isUserCreated` and `isPortfolioCreated`, ensuring that the BPMN gateways can always reach the modeled compensation paths.

3.1.7 Transactional Outbox

Lecture 5 discussed the need to solve inconsistent dual writes. CryptoFlow addresses this with the Transactional Outbox pattern in Section 6.14.

When an order is approved, `approveOrderWorker` stores both the `APPROVED` status and an outbox row in the same local database transaction of `transaction-service`. `publishOrderApprovedWorker` then publishes the pending outbox entry to Kafka and marks it as published. If the service crashes after the database commit but before publication, the scheduler republishes stale unpublished rows. This guarantees that an approved order does not silently miss the event that must later update the portfolio.

3.1.8 Idempotent Consumer

Lecture 5 also stressed that retries and redelivery are safe only if consumers are idempotent. CryptoFlow implements this in Section 6.16 for `portfolio-service`.

Before applying an `OrderApprovedEvent`, the service inserts the `transactionId` into the `processed_transaction` table under a unique constraint. If the insert succeeds, the holding update is executed in the same transaction. If the insert violates the constraint, the event has already been processed and is ignored. This protects the portfolio from duplicated Kafka deliveries after crashes, restarts, or rebalancing.

3.1.9 Replicated Read-Model

The lecture material on read models and CQRS motivated a local projection for autonomous validation. CryptoFlow applies that idea in Section 6.17.

`transaction-service` keeps its own table of confirmed users instead of calling `user-service` during order placement. `user-service` publishes `UserConfirmedEvent` records to a log-compacted topic, and `transaction-service` upserts the confirmation state into its local database. Order validation is therefore a local read. This is a focused, pragmatic use of a replicated read-model: the write model remains in `user-service`, while the trading context keeps only the projection it needs.

3.1.10 Human Intervention as a Stateful Resilience Pattern

Lecture 5 presented human intervention as a stateful resilience pattern for deterministic failures that should not be retried forever. CryptoFlow uses exactly this pattern in Section 6.18 for the `placeOrder` workflow.

If the approval step cannot find the expected transaction record, or if the publication step cannot find the expected outbox row, the worker throws a typed BPMN error. Boundary events route the instance to an operations user task in Camunda Tasklist, where the relevant order context is visible and the operator must document the resolution. This keeps the process instance open, auditable, and operationally visible instead of turning a known data-integrity problem into endless retries or a silent failure.

3.2 Event-driven Concepts

3.2.1 Streams as Unbounded, Replayable Event Logs

The second half of the lectures treated an event stream as an unbounded and replayable sequence of facts. This extends the earlier Kafka integration model: Kafka is no longer only a transport between services, but also the durable input for long-running computations.

CryptoFlow implements this by keeping raw input topics as replay boundaries. Binance ticker events are published to `crypto.price.raw` according to the existing market-data decision in Section 6.6. Binance partial book depth events are published to `crypto.scout.raw` according to Section 6.21. These raw topics let derived topologies rebuild their outputs after a fresh Kafka Streams `application.id`, an offset reset, or a local state-store deletion. Section 6.24 makes this explicit for Market Scout after the service split.

3.2.2 Stateless Stream Processing

The lectures introduced single-event stream transformations such as filter, translator, splitter, and router. These operations do not depend on previous events, so they scale horizontally and replay deterministically.

CryptoFlow uses this pattern in the Market Scout ingestion path. `market-partial-book-ingestion-service` captures raw partial book depth records, while `market-order-scout-service` filters out records without asks, splits ask levels into `AskQuote` records, translates quotes into the shared `MatchableAsk` contract, and filters threshold hits into `AskOpportunity` records. Section 6.23 separates raw ingestion from scout processing, Section 6.22 defines the scout-local Avro contracts, and Section 6.26 defines `MatchableAsk` as the cross-service ask contract for transaction matching.

The same stateless shape appears in Reference Data. `fx-rate-service` polls a provider, filters the supported currency pairs, translates them into `FxRate` events, and publishes them to the compacted `reference.fx.rate` topic as described in Section 6.29. `coin-metadata-service` does the same for slow-moving cryptocurrency metadata, publishing `CoinMetadata` records to `reference.crypto.metadata` as described in Section 6.33.

3.2.3 Stream-Table Duality and Local State

The lectures then introduced stream-table duality: a `KStream` represents facts over time, while a `KTable` represents the latest state obtained by applying those facts. Kafka Streams materializes those tables in local state stores backed by changelog topics.

CryptoFlow uses this in portfolio valuation. Section 6.34 places a Kafka Streams topology inside `portfolio-service`: approved orders are folded into a holdings table keyed by `userId|symbol`, prices are materialized by symbol, and the resulting position values are grouped into a per-user `portfolio-value-store`. The endpoint `GET /portfolios/{userId}/streams-value` reads that local store through interactive queries instead of issuing a synchronous request to another service.

Transaction matching also depends on local state. Section 6.27 replaces heap-based ticker matching with persistent stores for pending bids, matched transaction ids, and allocated ask

quote ids. This makes the matching decision restartable and replayable. A transaction that already matched is skipped on replay, and an ask quote can be allocated to at most one pending bid.

3.2.4 Event Time, Windows, Grace Periods, and Suppression

The windowing lectures distinguished event time from processing time and introduced grace periods for bounded late events. CryptoFlow applies event time wherever the business meaning depends on when the market event happened.

Market Scout aggregates `AskOpportunity` events into window summaries. The topology groups opportunities by symbol, applies a configured tumbling window, and emits `ScoutWindowSummary` records. This is part of the scout-local Avro event model from Section 6.22.

The OHLC topology is the more complete windowing implementation. Section 6.31 decides that OHLC bars are emitted venue-native in USDT on `crypto.ohlc.1m`, `crypto.ohlc.5m`, and `crypto.ohlc.1h`. The topology extracts event timestamps from price events, aggregates open/high/low/close values in tumbling event-time windows, applies a grace period, and uses `suppress(untilWindowCloses)` so downstream consumers receive one final closed candle rather than every intermediate update.

Transaction matching uses event-time validity as business logic rather than as a DSL windowed join. Section 6.27 defines a 30-second bid validity interval plus a five-second processing and retention margin aligned with the Camunda timeout. A `MatchableAsk` can match a `BuyBid` only if the ask's event time falls inside the 30-second business interval and price/quantity constraints pass.

3.2.5 Joins and Repartitioning

The second-half lectures covered joins between streams and tables, plus the repartitioning needed when a topology changes keys. CryptoFlow implements these concepts in multiple places.

The FX localisation design in Section 6.30 uses a stream-table join: `crypto.price.clean` is joined with the compacted `reference.fx.rate` table to produce `LocalizedPrice` events. This is an ADR-locked target design; the currently implemented valuation and OHLC topologies still read `crypto.price.raw`. The implemented OHLC flow uses the same pattern family for metadata enrichment. Section 6.33 materializes `reference.crypto.metadata` as a `GlobalKTable` and left-joins it into closed OHLC bars so dashboard consumers receive names, logo URLs, base/quote assets, and ranking data without synchronous metadata lookups.

Portfolio valuation demonstrates table-table joining and multiphase repartitioning. Section 6.34 joins the holdings table with the latest price table by symbol. The intermediate result is keyed by `userId|symbol`, which is correct for per-position values. The dashboard needs a per-user sum, so the topology groups by `userId`, triggering the repartition step that moves all positions for one user to the same task before aggregating into `portfolio-value-store`.

3.2.6 Event Contracts and Schema Evolution

The stream-processing lectures also emphasized that derived streams need explicit contracts. CryptoFlow therefore uses different serialization strategies depending on ownership and coupling.

Raw replay topics remain JSON, following Section 6.3 and the raw-boundary decision in Section 6.22. This keeps `crypto.price.raw` and `crypto.scout.raw` easy to inspect and replay during demonstrations.

Scout-local derived topics use registryless Avro. `AskQuote`, `AskOpportunity`, and `ScoutWindowSummary` are owned by `market-order-scout-service`, so Section 6.22 keeps them as local generated Avro classes without adding Schema Registry infrastructure. `MatchableAsk` is a deliberate boundary case: Section 6.26 places it in `shared-events` because it crosses into the Trading bounded context, but the shipped implementation still serializes it with the local registryless `SpecificAvroSerde` rather than Confluent Schema Registry.

New cross-service derived contracts use Confluent Schema Registry. Section 6.32 introduces registry-backed Avro for events such as `FxRate`, `LocalizedPrice`, `PortfolioValue`, `Ohlc`, `CoinMetadata`, and `UserDisplayCurrencyUpdated`. In the shipped slice, `FxRate`, `PortfolioValue`, `Ohlc`, `CoinMetadata`, and `UserDisplayCurrencyUpdated` are active registry-backed contracts, while `LocalizedPrice` remains the schema-managed target for ADR-0030. This gives independently deployable producers and consumers a central compatibility boundary while preserving JSON at raw replay points.

4 Architecture

This chapter describes the system architecture of CryptoFlow. It begins with the service topology and the two main end-to-end flows, then covers the event-driven layer (Kafka topology), the process-oriented layer (BPMN workflows), the software-level architectural decisions, and closes with a summary of key architectural decision records.

4.1 Service Topology

CryptoFlow is a distributed system comprising nine Spring Boot microservices that communicate exclusively through Apache Kafka events and Camunda 8 (Zeebe) process orchestration. No service makes synchronous REST or gRPC calls to another service for domain operations. The REST endpoints are exposed only for external client access.

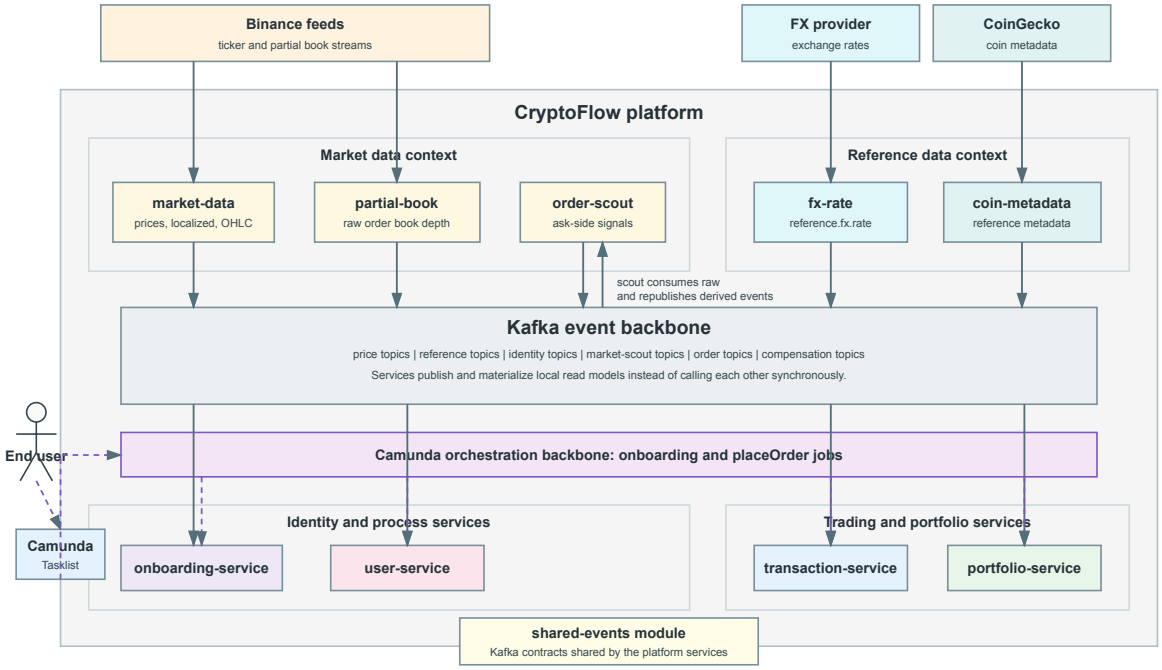


Figure 3: Simplified platform topology showing deployable services grouped around the Kafka event backbone and Camunda orchestration backbone

Figure 3 expands the high-level context map (Figure 1) and deployment overview (Figure 2) from Chapter 1. It intentionally groups the deployable services around two integration backbones instead of drawing every topic-level edge: Kafka carries the domain events and materialized read-model updates, while Camunda coordinates the onboarding and `placeOrder` workflows. The deployable services shown in the topology have the following responsibilities.

Market Data Service Inbound adapter to Binance ticker streams. Publishes `CryptoPriceUpdatedEvent` records, hosts OHLC stream processing, and owns no database. The price-localization stream in ADR-0030 remains a documented target

shape, while the implemented OHLC and portfolio valuation flows currently read `crypto.price.raw`.

Market Partial Book Ingestion Service Inbound adapter to Binance partial book streams. Publishes replayable raw order-book depth events to `crypto.scout.raw`.

Market Order Scout Service Kafka Streams processor for the market-scout pipeline. Consumes `crypto.scout.raw`, derives ask-side scout events, and publishes matchable asks for trading.

FX Rate Service Reference-data producer that polls an external FX provider and publishes compacted `reference.fx.rate` events for display-currency conversion.

Coin Metadata Service Reference-data producer that polls external coin metadata and publishes compacted metadata events for market-data enrichment.

Portfolio Service Maintains portfolio and holding state in PostgreSQL. It keeps a local in-memory price cache derived from `crypto.price.raw`, exposes REST endpoints for valuation queries, updates holdings from approved-order events, and acts as a Camunda job worker for portfolio creation during onboarding.

Transaction Service Owns the trading bounded context (ADR-0020). It deploys the `placeOrder.bpmn` process, validates incoming orders against a replicated read-model of confirmed users, matches pending buy bids against Matchable Asks in a Kafka Streams topology, and publishes approved-order events via its transactional outbox for downstream portfolio updates.

User Service Owns user accounts, confirmation links, and confirmation-state changes. It acts as a Camunda job worker for user preparation, creation, and invalidation during onboarding, publishes confirmed-user events for downstream consumers, and handles compensation by deleting users when the onboarding saga fails.

Onboarding Service Dedicated saga orchestrator (ADR-0010) that deploys `userOnboarding.bpmn` to Zeebe and coordinates the registration flow across the user and portfolio contexts. It owns no persistent business data because the workflow state lives in the Zeebe engine.

The `shared-events` module shown in Figure 3 is not a runtime service. It is a shared Maven module that defines the Kafka event contracts consumed and produced by the participating services, ensuring compile-time consistency across the codebase (ADR-0005).

Supporting infrastructure includes a Kafka broker (KRaft mode), PostgreSQL 16, Kafka UI for monitoring, and pgAdmin for database administration, all provisioned via Docker Compose.

4.1.1 Onboarding Flow

The onboarding flow spans three services and one external system. The user fills out a registration form in Camunda Tasklist, which starts the `userOnboarding` process instance in Zeebe. Inside `user-service`, the `prepareUserWorker` generates a `userId`, persists a `PENDING` confirmation link, and builds the confirmation email content. The Camunda email connector sends the email via SMTP. The process then waits at an event-based gateway for either a confirmation click or a one-minute timeout.

If the user clicks the confirmation link, `user-service` receives the HTTP request on its `UserConfirmationController`, marks the link as `CONFIRMED`, publishes a `UserConfirmedEvent` to Kafka, and correlates a `Message_UserConfirmed` message to Zeebe. The process then fans out through a parallel gateway: `userCreationWorker` creates the user record in `user-service`'s database, while `portfolioCreationWorker` creates a portfolio record in `portfolio-service`'s database. Both branches must succeed for the saga to complete. If either fails, compensation handlers delete the already-created entity and publish a compensation event for the peer service (ADR-0011). If the timer fires instead, the `invalidateConfirmationWorker` marks the link as `INVALIDATED` and the process ends.

4.1.2 Trading Flow

The trading flow stays primarily within the trading bounded context, but the market decision is now fed by the Market Scout stream. A user submits an order through Camunda Tasklist, starting a `placeOrder` process instance. The `order-processing-worker` validates the user against the local confirmed-user read-model (ADR-0017), creates a `PENDING` transaction record, and publishes a `BuyBid` to `transaction.buy-bids`. In parallel, Market Scout publishes `MatchableAsk` records derived from Binance partial-book snapshots to `crypto.scout.matchable-asks`. A Kafka Streams topology in `transaction-service` matches bids and asks by symbol and emits `transaction.order-matched` when the event-time, price, quantity, and allocation rules pass.

The process waits at an event-based gateway for either an order-matched event (correlated by `transactionId`) or a timeout. On a match, the `approveOrderWorker` writes both the `APPROVED` status and an outbox row in a single database transaction (ADR-0014). The `publishOrderApprovedWorker` then publishes the outbox entry to Kafka and marks it as published. The Camunda email connector notifies the user that the order was executed.

The portfolio update happens outside the orchestrated flow: `portfolio-service` independently consumes the `OrderApprovedEvent` from Kafka and updates holdings via `upsertHolding()`, protected by an idempotent consumer guard (ADR-0016). This separation

is deliberate — the order approval is the terminal business event in the trading context, and the portfolio propagation is an eventual downstream consequence (ADR-0013, ADR-0015).

If the `approveOrderWorker` or `publishOrderApprovedWorker` encounters a deterministic failure (e.g., a missing transaction record or outbox row), it throws a typed BPMN error that routes the instance to an operations user task for human investigation (ADR-0018).

4.2 Event-Driven Architectural Parts

Domain events, reference-data updates, and derived stream-processing events flow through Kafka. Figure 4 visualizes the current topology as implemented in the codebase, and Table 4 summarizes the topic-level configuration.

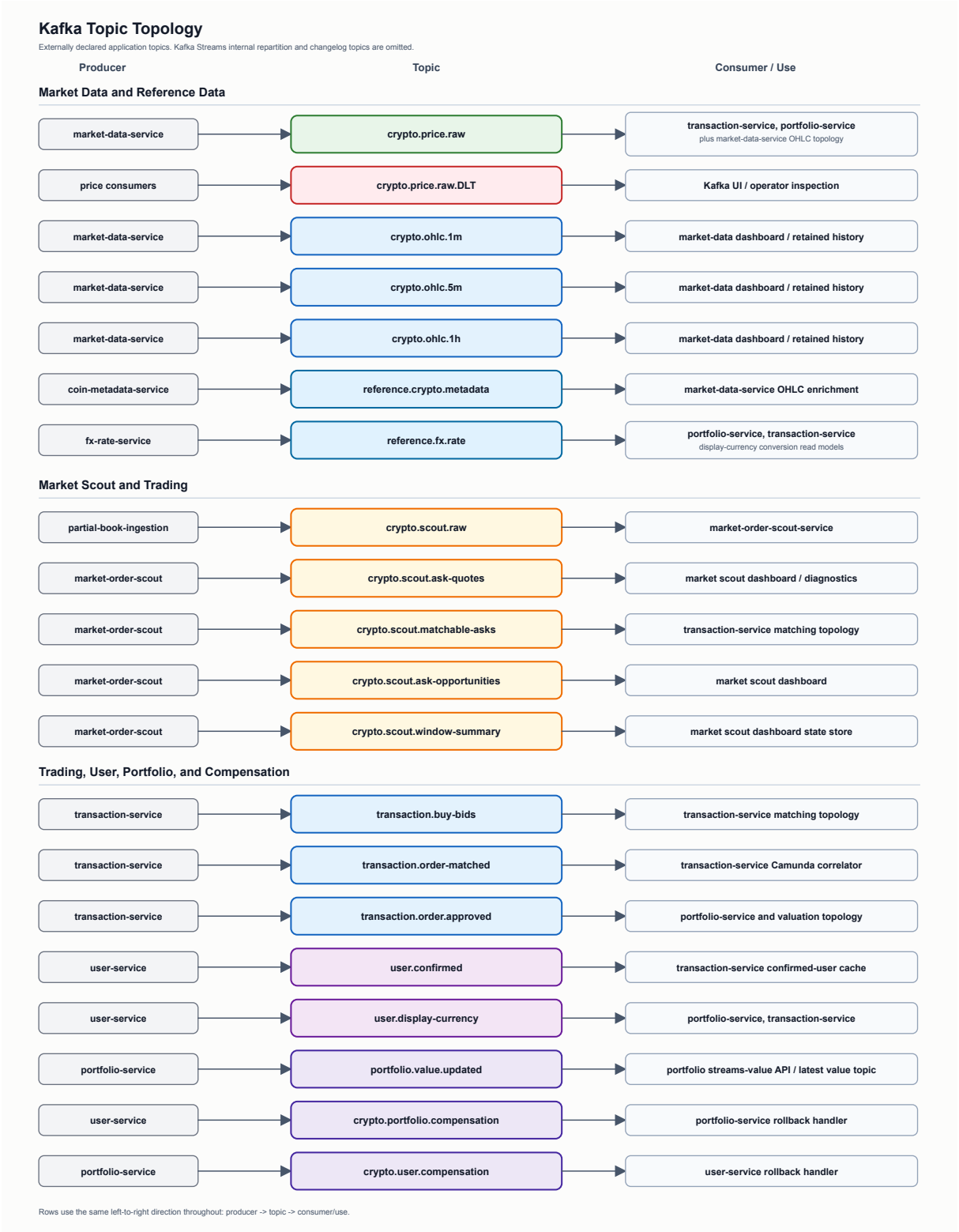


Figure 4: Kafka topic topology rendered from the current implementation

Topic	Partitions	Retention / cleanup	Message key	Purpose
crypto.price.raw	3	1 hour	symbol	Live Binance ticker prices
crypto.price.raw.DLT	1	1 hour	Original key	Dead-letter stream for failed raw price records
transaction.order.approved	3	1 hour	transactionId	Approved orders consumed by Portfolio and valuation streams
transaction.buy-bids	3	1 hour	symbol	Buy bids emitted by the place-order worker for matching
transaction.order-matched	3	1 hour	transactionId	Matching decisions correlated back into Camunda
user.confirmed	3	Log-compacted	userId	Confirmed-user read-model for Trading
user.display-currency	1	Log-compacted	userId	Per-user Display Currency read-model for Portfolio and Trading
reference.fx.rate	1	Log-compacted	currencyPair	FX-rate reference table
reference.crypto.metadata	1	Log-compacted	symbol	Coin metadata for OHLC enrichment
crypto.scout.raw	3	5 min / size-bounded	symbol	Raw Binance partial-book depth events
crypto.scout.ask-quotes / matchable-asks / ask-opportunities / window-summary	3	5 min / size-bounded	symbol	Market Scout derived streams, including the cross-service ask contract for Trading
crypto.ohlc.1m / 5m / 1h	3	7 / 30 / 365 days	symbol	Closed OHLC bars emitted after window close and grace
portfolio.value.updated	3	Log-compacted	userId	Current portfolio value from the valuation topology
crypto.portfolio.compensation / crypto.user.compensation	3	1 hour	userId	Bidirectional onboarding rollback requests between User and Portfolio

Table 4: Kafka topic configuration

The producer/consumer relationships are:

Service	Produces to	Consumes from
Market Data	crypto.price.raw, crypto.ohlc.*	crypto.price.raw, reference.crypto.metadata
Partial Book Ingestion	crypto.scout.raw	–
Market Order Scout	crypto.scout.ask-quotes, crypto.scout.matchable-asks, crypto.scout.ask-opportunities, crypto.scout.window-summary	crypto.scout.raw
FX Rate	reference.fx.rate	–
Coin Metadata	reference.crypto.metadata	–
Portfolio	crypto.user.compensation, portfolio.value.updated	crypto.price.raw, transaction.order.approved, crypto.portfolio.compensation, reference.fx.rate, user.display-currency
Transaction	transaction.buy-bids, transaction.order-matched, transaction.order.approved	crypto.price.raw, transaction.buy-bids, crypto.scout.matchable-asks, transaction.order-matched, user.confirmed, reference.fx.rate, user.display-currency
User	crypto.portfolio.compensation, user.confirmed, user.display-currency	crypto.user.compensation

Table 5: Kafka producers and consumers by service

Table 4 captures the topic-level configuration, while Table 5 shows which service publishes and consumes each stream. The onboarding service is intentionally absent because it coordinates via Zeebe rather than Kafka. Topic cleanup is deliberately mixed: raw and matching streams use short delete retention, reference data and current-value streams are compacted caches, and OHLC bars retain longer interval-specific histories. The partition key strategy is topic-specific: market and scout streams use normalized symbol keys; `transaction.order.approved` and `transaction.order-matched` use `transactionId`; user-related topics use `userId`; FX rates use directed currency-pair keys such as `USDCHF`; and the DLT preserves the failed record's original Kafka key.

4.2.1 Broker, Producer, and Consumer Configuration

Beyond the topic layout, the Kafka layer is configured at three levels: broker, producer, and consumer. Table 6, Table 7, and Table 8 summarize the default settings and the deliberate deviations introduced by the second-half stream-processing work.

Property	Value	Rationale
`process.roles`	`broker,controller`	Single-node KRaft mode — no Zookeeper dependency
`auto.create.topics.enable`	`false`	Topics are declared programmatically via Spring `NewTopic` beans, preventing typos or misconfigured producers from silently creating unintended topics
`offsets.topic.replication.factor`	`1`	Required for single-broker operation
`log.retention.hours`	`1`	Sufficient for real-time price streaming; consumers that fall behind lose events. Domain state is persisted in PostgreSQL, so Kafka is not the system of record
`log.retention.bytes`	`100 MB`	Per-partition cap that prevents disk exhaustion in development
`log.cleanup.policy`	`delete`	Default for raw and transient streams. Reference-data and current-value topics override to `compact` at topic level

Table 6: Kafka broker configuration (Docker Compose)

Scope / Property	Value	Rationale
Default producer `acks`	`all`	Market-data, partial-book, reference-data, and Avro producers request all in-sync replicas. With a single broker this is functionally equivalent to `acks=1`, but the setting is forward-compatible with a replicated deployment (ADR-0001)

Default key serializer	`StringSerializer`	Message keys are plain strings: symbols, user ids, transaction ids, or directed currency-pair keys
Legacy/domain value serializer	`JsonSerializer`	Raw replay topics and original domain events remain readable JSON without type headers (ADR-0003)
Derived cross-service value serializer	`KafkaAvroSerializer` / `SpecificAvroSerde`	`FxRate`, `CoinMetadata`, `Ohlc`, `PortfolioValue`, and `UserDisplayCurrencyUpdated` use Avro with Schema Registry (ADR-0032)
Scout-local derived serializer	Registryless Avro serde	`AskQuote`, `AskOpportunity`, `ScoutWindowSummary`, and the shared `MatchableAsk` contract use generated Avro classes without Schema Registry

Table 7: Kafka producer configuration

Scope / Property	Value	Rationale
Listener offset reset	`earliest`	New consumer groups replay from the oldest retained record. Validated in the consumer offset experiments (Chapter 7): `latest` silently skips retained history
Listener auto commit	`false`	Offsets are committed after processing, not on poll. Prevents the consumer from advancing past records it has not yet processed
JSON price listeners	`ErrorHandlingDeserializer` + DLT	Malformed price records are caught at deserialization and, after retries, published to `crypto.price.raw.DLT` so one poison pill does not block the partition
Compensation listeners	JSON deserializer + retry	Compensation events are idempotent and retryable; they do not currently route to a DLT
Avro/reference listeners	`KafkaAvroDeserializer`	FX-rate and display-currency consumers use dedicated Schema Registry-aware factories
Kafka Streams apps	`earliest` + `LogAndContinueExceptionHandler`	A fresh Streams application can rebuild retained state, while deserialization failures in stream tasks are logged and skipped instead of blocking the topology

Table 8: Kafka consumer configuration

The configuration is intentionally not uniform anymore. The first-half services still use JSON for raw and domain events, while the second-half additions introduced Schema Registry-backed Avro for cross-service derived events and registryless Avro for scout-local derived streams. The same distinction appears on the consumer side: listener containers, Avro consumers, and Kafka Streams applications each use the error-handling model that fits their topic ownership and replay semantics.

Single-broker trade-off. The current deployment runs a single Kafka broker with no replication. This means that `acks=all` provides no durability advantage over `acks=1`. The experiments in Chapter 7 demonstrate this directly: data loss under `acks=1` only occurs when a leader crashes before followers replicate, and with one broker there are no followers. The configuration is a conscious development-environment choice (ADR-0001): the architecture and code are designed for a replicated cluster, but the Docker Compose stack prioritizes simplicity over fault tolerance. A production deployment would require a multi-broker cluster with `replication.factor >= 3` and `min.insync.replicas = 2` to realize the durability guarantees that `acks=all` is designed to provide.

4.3 Process-Oriented Architectural Parts

CryptoFlow uses Camunda 8 / Zeebe (ADR-0008) to orchestrate the two long-running business processes described above. This section shows the BPMN models and explains the modeling decisions behind them.

4.3.1 Commands, Events, and Workers

CryptoFlow distinguishes three interaction primitives in its BPMN models, each with a fixed naming convention and distinct runtime semantics summarized in Table 9. Commands and workers share an imperative naming style but differ at runtime: Zeebe invokes a worker over gRPC and halts the instance on failure so the error can be caught by a boundary event (ADR-0018, see Figure 8), whereas a command dispatches a message and the downstream consumer acts autonomously without reporting back to the engine. This convention is applied consistently in both `userOnboarding.bpmn` and `placeOrder.bpmn`.

Primitive	BPMN element	Naming convention	Runtime semantics
Command	Send task	verb + subject (imperative); job type suffix `Command`	Orchestrator dispatches a message and continues. Downstream consumer acts autonomously; Zeebe receives no acknowledgement
Event	Message intermediate catch event	subject + verb (past tense); message reference suffix `Event`	Correlated fact the process waits for. Zeebe durably suspends the instance and resumes it when a matching message arrives
Worker	Service task	verb + subject (imperative); job type suffix `Worker`	Zeebe invokes the worker over gRPC with retries. Deterministic failures halt the instance at that step and can be caught by boundary events

Table 9: Naming conventions and runtime semantics for BPMN interaction primitives

4.3.2 User Onboarding Process

The `userOnboarding.bpmn` process, deployed by the onboarding service, implements a Parallel Saga (ADR-0010) that coordinates user and portfolio creation across two bounded contexts.

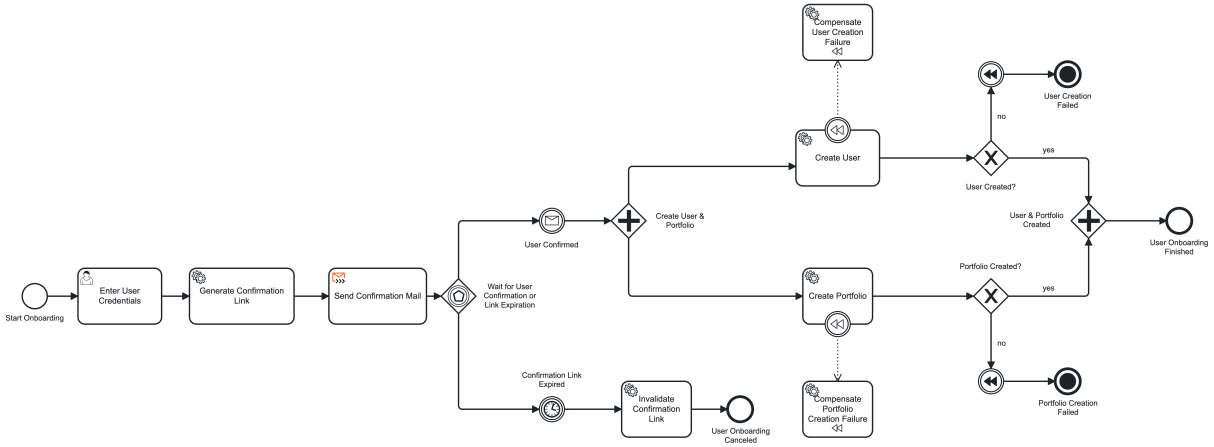


Figure 5: User onboarding BPMN process with parallel saga, confirmation wait, and compensation paths

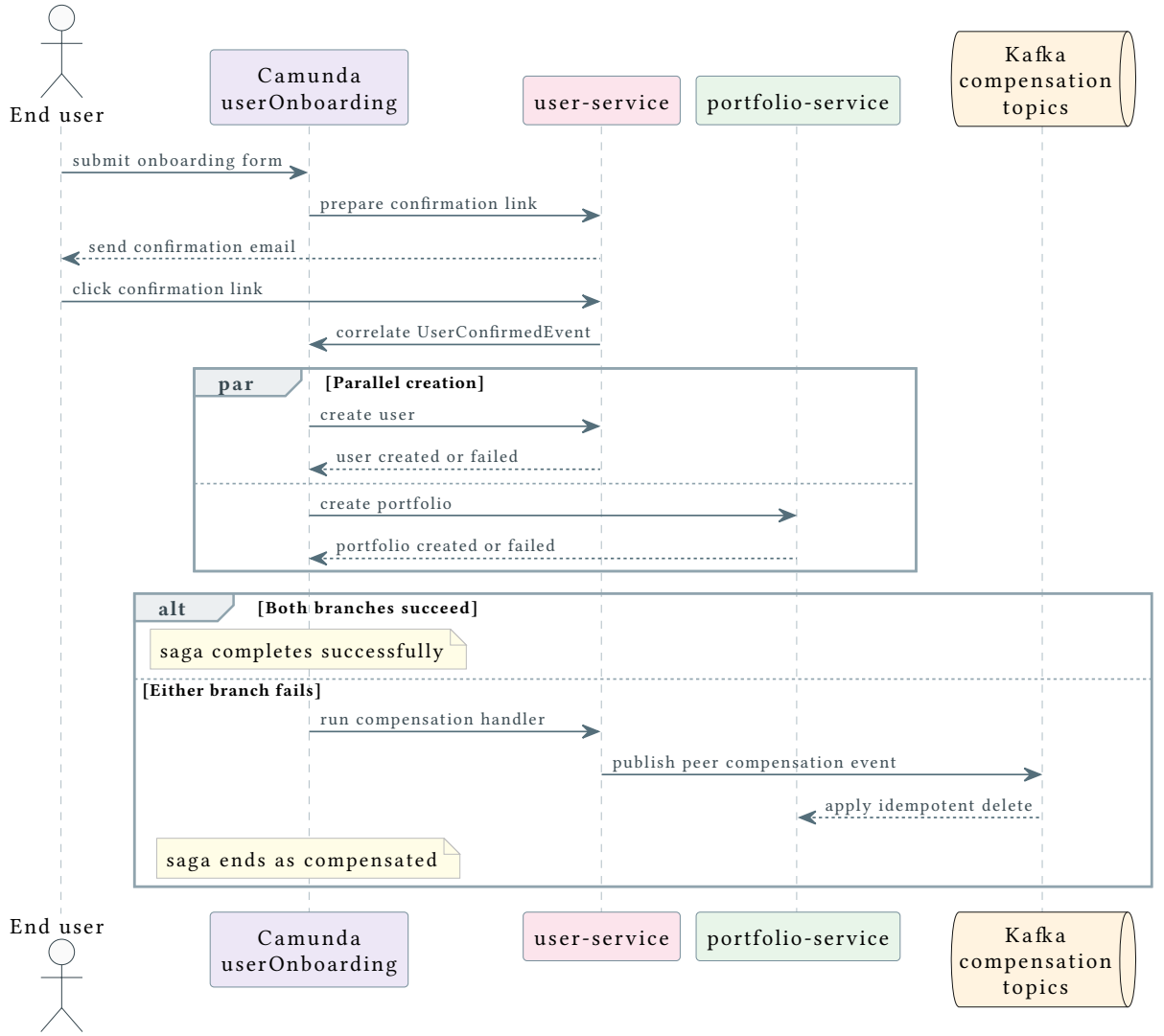


Figure 6: Onboarding runtime sequence: confirmation, parallel creation, and compensation. The compensation branch is drawn in one direction; the mirror path applies when the user-creation branch is the failing one

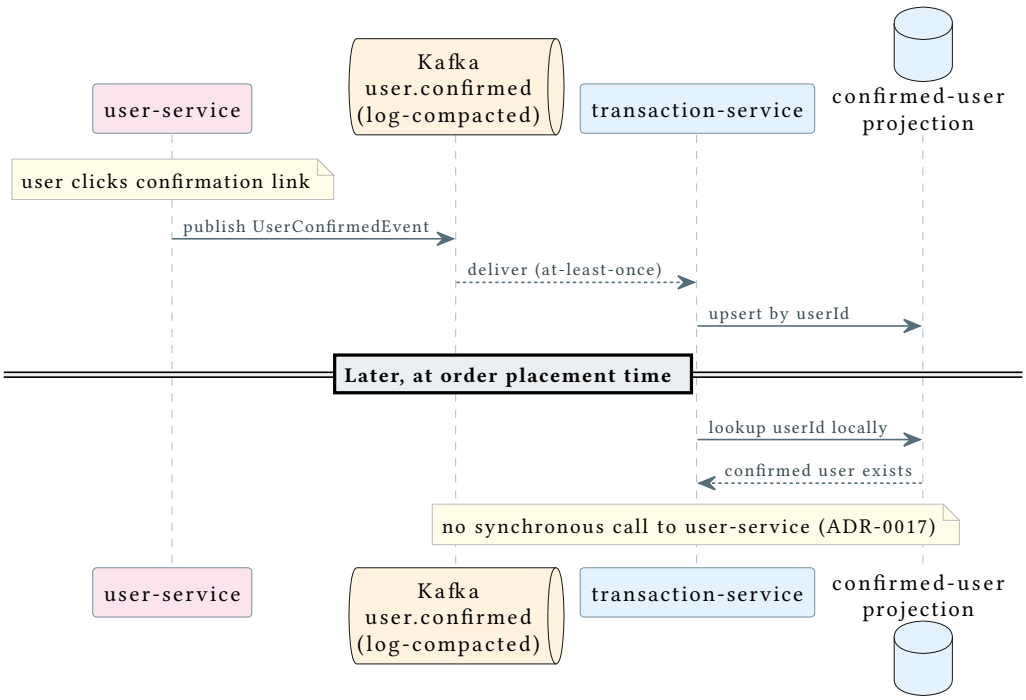


Figure 7: Supporting detail: `UserConfirmedEvent` propagation into the `transaction-service` confirmed-user projection, enabling later order placement to validate users locally (ADR-0017)

Figure 5 shows the orchestrated BPMN shape, while Figure 6 shows the core runtime collaboration. A parallel creation and compensation mediated by the compensation Kafka topics (ADR-0011). The cross-cutting read-model propagation triggered by the confirmation step is kept out of Figure 6 to preserve focus; Figure 7 shows it as a dedicated detail. Its structure reflects three key modeling decisions:

Event-based confirmation wait. After the confirmation email is sent, the process suspends at an event-based gateway rather than polling or blocking a thread. Zeebe durably persists the waiting state, so the process can survive service restarts during the wait. The one-minute timer boundary ensures that abandoned registrations terminate cleanly by marking the confirmation link as `INVALIDATED` (ADR-0010).

Parallel gateway for cross-context creation. After confirmation, a parallel gateway fans out to `userCreationWorker` (user-service) and `portfolioCreationWorker` (portfolio-service). Both execute their own local ACID transaction in their own bounded context. The saga proceeds only when both branches report their outcome. This models the Parallel Saga pattern from Lecture 5: communication is asynchronous, consistency is eventual, and coordination is centralized in one orchestrator (ADR-0010).

Flag-driven completion and compensation branches. Both workers always complete their Zeebe jobs successfully and communicate the outcome through boolean flags (`isUserCreated`, `isPortfolioCreated`) rather than throwing BPMN errors. This is formalized

in ADR-0012: throwing errors would short-circuit the flow and bypass the exclusive gateways that route to the compensation paths modeled in ADR-0011. If either flag is `false`, the BPMN routes to compensation handlers that delete the already-created entity and publish an event-carried compensation request for the peer service. The compensation events carry all necessary state (userId, userName, email, portfolioId) so the receiving service can execute the rollback without a synchronous callback.

4.3.3 Place Order Process

The `placeOrder.bpmn` process (see Figure 8), deployed by the transaction service, implements a Fairy Tale Saga (ADR-0013) that keeps synchronous service tasks inside the trading context but accepts eventual consistency at the boundary to the portfolio context.

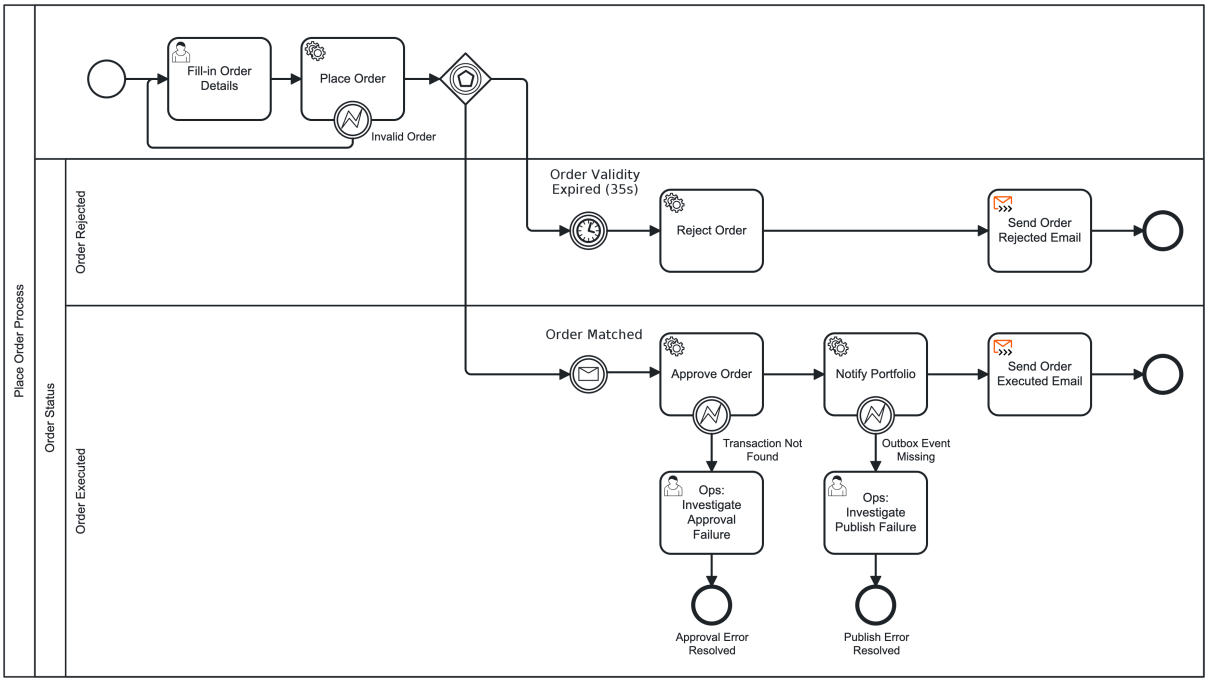


Figure 8: Place order BPMN process with Kafka-backed matching, timeout handling, and downstream publication

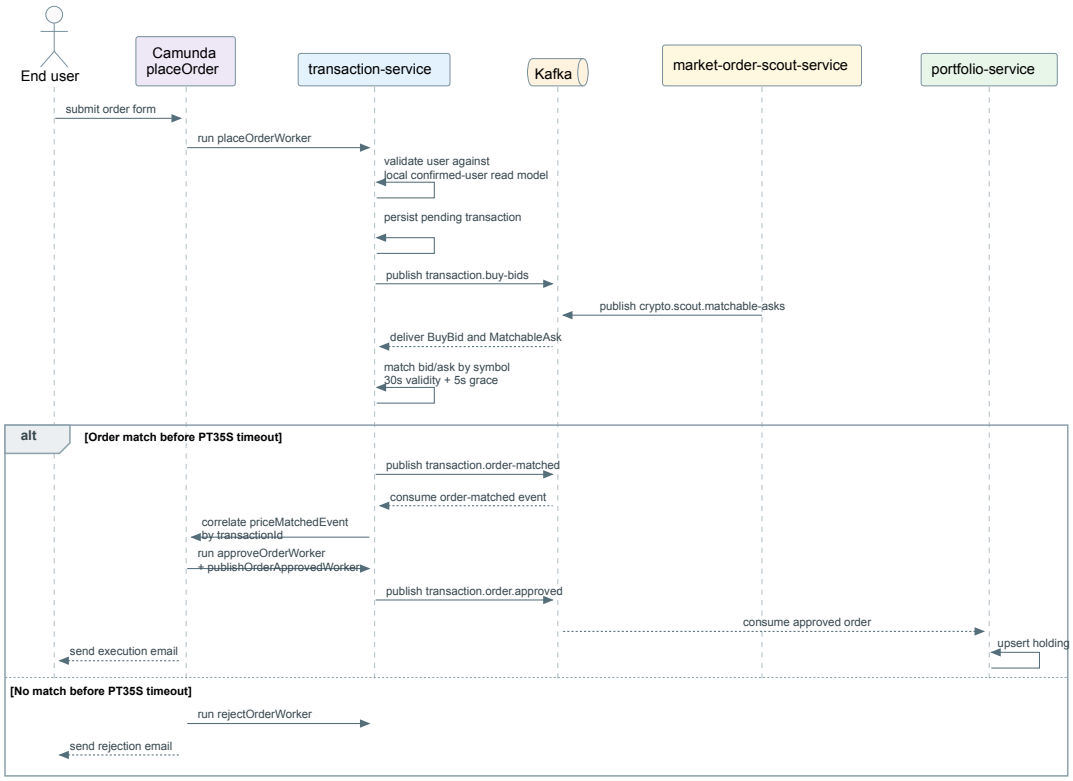


Figure 9: Place order runtime sequence: local user validation, Kafka-backed matching wait, approval, publication, and downstream portfolio update

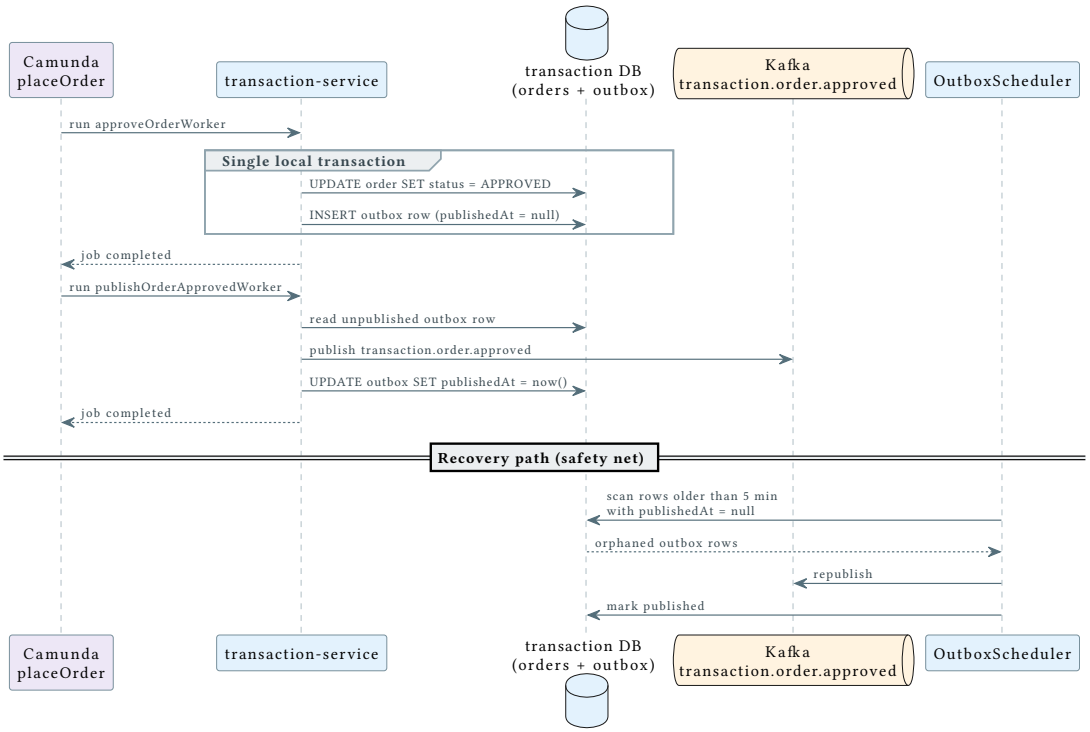


Figure 10: Supporting detail: transactional outbox mechanics. `approveOrderWorker` writes the APPROVED status and the outbox row in one local transaction; `publishOrderApprovedWorker` publishes and marks the row; the `OutboxScheduler` republishes orphaned rows (ADR-0014)

Figure 8 shows the orchestrated BPMN shape, while Figure 9 shows the core runtime collaboration. This includes the downstream `portfolio-service` consumer that is deliberately outside the orchestrated flow (ADR-0013, ADR-0015) and therefore not visible in the BPMN. The outbox write/publish/recover mechanics are kept out of Figure 9 to preserve focus; Figure 10 shows them as a dedicated detail. Its structure reflects the following modeling decisions:

Event-based matching wait. The process suspends at an event-based gateway while the transaction matching topology waits for a compatible `MatchableAsk`. When `transaction-service` consumes a `transaction.order-matched` event, it correlates the Camunda message by `transactionId`. This wait is of non-deterministic duration. It may last seconds or never terminate. Zeebe durably suspends the instance without holding a distributed lock or database connection (ADR-0013). A 35-second timer mirrors the 30-second matching window plus a five-second processing and retention margin: if no match occurs, the order is rejected and the user is notified.

4.3.3.1 Matching Extension

The original `placeOrder` flow matched pending orders against the latest ticker price held inside `transaction-service`. The stream-processing extension keeps the BPMN shape and the Fairy Tale Saga boundary intact, but replaces that simplified price check with a bid/ask matching pipeline. A placed order becomes a `BuyBid` event, `market-order-scout-service` supplies `MatchableAsk` events derived from Binance partial-book data, and a Kafka Streams topology in `transaction-service` emits `transaction.order-matched` when a bid can be allocated to an ask within the configured event-time validity window.

At the architecture level, the important point is separation of concerns: Kafka Streams owns the market-event matching decision, while Camunda still owns the business workflow that approves, rejects, notifies, and publishes the approved-order event through the outbox. The detailed topology, state stores, event-time rules, and allocation semantics are described in Chapter 5.

Transactional outbox for reliable publication. The approval path uses two sequential workers. `approveOrderWorker` writes both the `APPROVED` status and an outbox row in a single local database transaction. `publishOrderApprovedWorker` then reads and publishes the outbox entry to Kafka. A scheduled safety net republishes any rows that remain unpublished after a crash (ADR-0014). This guarantees that an approved order does not silently miss the event that must later update the portfolio.

Human escalation for deterministic failures. If the approval or publication step encounters a deterministic, non-retryable failure (e.g., a missing transaction record or outbox row), the worker throws a typed BPMN error. An interrupting boundary event routes the instance to an operations user task in Camunda Tasklist, where the operator sees the full order context and must document the resolution before closing the task (ADR-0018). This keeps the instance open and auditable instead of retrying indefinitely or failing silently.

Eventual portfolio propagation. The portfolio update is deliberately not part of the orchestrated flow. After the outbox event is published to Kafka, `portfolio-service` consumes `OrderApprovedEvent` independently and updates holdings in its own ACID boundary, protected by an idempotent consumer guard (ADR-0016). The executed email confirms the order approval — the business event — not the portfolio propagation, which is a downstream consequence accepted as eventually consistent (ADR-0015).

4.4 Software Architectural Parts

This section covers the software-level decisions that support the event-driven and process-oriented architecture above. These patterns were already familiar from prior coursework (ASSE) and are therefore treated concisely.

4.4.1 Microservice Decomposition

CryptoFlow is implemented as independently deployable Spring Boot 3.5 services on Java 21. Each service belongs to a bounded context and communicates with other services exclusively through Kafka and Zeebe. REST endpoints are exposed only for external client access, not for inter-service calls (ADR-0001).

4.4.2 Hexagonal Architecture

The services follow the same internal structure where applicable, separating the business core from technical adapters:

- `domain/` contains the domain model and invariants.
- `application/` contains use-case logic and the ports that the core depends on.
- `adapter/in/` contains inbound adapters: REST controllers, Kafka listeners, Camunda job workers, or the Binance WebSocket entry.
- `adapter/out/` contains outbound adapters: JPA repositories, Kafka producers, and external clients.

This separation ensures that transport, workflow-engine, and persistence details remain replaceable without moving domain rules into framework code.

4.4.3 Data Ownership and Persistence

Each stateful service owns a dedicated PostgreSQL 16 database (ADR-0019): `user-service` owns `user_service_db`, `portfolio-service` owns `portfolio_service_db`, and `transaction-service` owns `transaction_service_db`. No service reads from or writes to another service's tables. Cross-service references use opaque identifiers, not foreign keys.

Schema changes are managed through Flyway versioned migrations. Hibernate runs in `validate` mode. This means that it verifies the entity-to-schema mapping at startup but never modifies the database (ADR-0007). Cross-service consistency is achieved through the patterns described above: Kafka events, replicated read-models, the transactional outbox, and orchestrated sagas.

4.4.4 Shared Event Contracts

Event schemas are defined once in the `shared-events` Maven module (ADR-0005). All producing and consuming services depend on it at compile time, ensuring type-safe event contracts. The module contains Java records, Avro schemas, and serialization dependencies but no business logic. Schema changes are atomic: a single commit updates the contract and all affected consumers. New cross-service derived events use Avro with Confluent Schema Registry per ADR-0032; raw replay topics remain JSON.

4.5 Key Architectural Decisions

The repository contains thirty-five accepted Architectural Decision Records (ADRs) in `docs/adrs`, excluding the template record. The most significant decisions are:

- **ADR-0001**: Kafka as the sole inter-service communication channel — no synchronous REST between services.
- **ADR-0002**: Event-Carried State Transfer for price data, enabling the portfolio service to answer queries without calling the market data service.
- **ADR-0008**: Camunda 8 (Zeebe) for process orchestration, chosen for its partitioned log, stateless worker model, and connector templates that keep Kafka and SMTP integration inside the BPMN model.
- **ADR-0010**: A dedicated onboarding service as saga orchestrator, separating the cross-context coordination concern from the participating services.
- **ADR-0011**: Event-carried compensation via Kafka for cross-service rollback, maintaining loose coupling even during failure handling.
- **ADR-0012**: Flag-driven completion for Camunda workers, ensuring BPMN compensation branches are always reachable.

- **ADR-0013**: Fairy Tale Saga for the placeOrder workflow, accepting eventual cross-service consistency to avoid distributed locks over a non-deterministic wait.
- **ADR-0014**: Transactional Outbox for reliable publication of approved-order events, preventing silent data loss between database commit and Kafka publication.
- **ADR-0017**: Replicated read-model for autonomous user validation at order placement, avoiding synchronous cross-service calls.
- **ADR-0018 / ADR-0035**: Human escalation for deterministic workflow failures, amended so operations resolution loops back into the `placeOrder` happy path instead of terminating the process.
- **ADR-0019**: Database per service for stateful bounded contexts, preserving independent deployment and schema ownership.
- **ADR-0020**: Transaction-service as sole owner of the trading bounded context, keeping order execution concerns isolated from user and portfolio management.
- **ADR-0021 to ADR-0025**: Market Scout uses Binance Partial Book Depth Streams as its replayable raw input, keeps raw events as JSON, derives ask-side Avro events, splits ingestion from scout processing, accepts a new Kafka Streams application id after the split, and enriches derived ask events with best-ask and notional features.
- **ADR-0026 / ADR-0027**: `MatchableAsk` is the narrow cross-service Avro contract from Market Scout to Trading, and bid/ask matching is implemented as a transaction-service Kafka Streams topology with a 30-second event-time validity window and price-time priority.
- **ADR-0028 to ADR-0030**: Display Currency is user-owned presentation data propagated through a compacted Kafka topic; FX rates live in a Reference Data bounded context; localized prices are the ADR-locked stream-table join target once the clean-price stream is available.
- **ADR-0031 / ADR-0033**: OHLC bars are emitted venue-native in USDT as closed bars and enriched with coin metadata through a GlobalKTable join, leaving display-currency conversion to read time.
- **ADR-0032**: New cross-service derived events use Avro with Confluent Schema Registry, while established raw JSON topics and registryless Market Scout Avro topics remain unchanged.
- **ADR-0034**: Portfolio valuation runs as a Kafka Streams topology inside `portfolio-service`, rebuilding holdings and values from event streams and exposing an interactive-query endpoint alongside the Postgres-backed read model.

5 Kafka Streams Extensions

This chapter describes the second-half stream-processing extension of CryptoFlow. The first report chapters describe an event-driven microservice system where Kafka transports facts between bounded contexts (Section 6.1). The extension work adds continuously running Kafka Streams topologies that filter, translate, join, window, aggregate, and materialize those facts into new event streams and queryable local state.

The structure follows the lecture progression from event streams to stateful and windowed topologies, then maps the concepts to the implemented services. The main implementation areas are Market Scout and transaction matching on Cyril's side, and reference data, OHLC aggregation, and portfolio valuation on Ioannis' side.

5.1 From Event Streams to Stream Processing Topologies

Lecture 07 framed an event stream as an unbounded, ordered, immutable, and replayable sequence of facts. This changes the design problem: the system does not first store all data and later run a finite query. Instead, the query is a long-running topology that continuously receives events, updates local state if needed, and emits derived events.

CryptoFlow uses that model in three places:

Source topics External systems are converted into Kafka topics. Binance ticker streams feed `crypto.price.raw` (Section 6.6), Binance partial book depth streams feed `crypto.scout.raw` (Section 6.21), and scheduled HTTP reference-data pollers feed the compacted `reference.fx.rate` and `reference.crypto.metadata` topics (Section 6.29, Section 6.33).

Processing topologies Kafka Streams applications transform those topics into derived topics. Market Scout derives ask quotes and opportunities (Section 6.22), Transaction Service matches bids with asks (Section 6.27), Market Data Service builds OHLC bars (Section 6.31), and Portfolio Service computes current portfolio values (Section 6.34).

Materialized views Some topologies persist local state stores. These stores are not side databases owned by another service; they are rebuildable views derived from Kafka input topics and their changelogs. Portfolio value and Market Scout dashboard state are exposed through interactive queries.

The stream-table duality is the common mental model. A `KStream` represents changes over time. A `KTable` represents the latest state after those changes have been applied. CryptoFlow uses both forms: prices become a latest-price table, holdings become a holdings table, FX and metadata topics are compacted reference tables, and windowed OHLC stores hold finite event-

time aggregates. Compacted topics are deliberately used as distributed caches for slow-moving facts such as FX rates, coin metadata, user display currency, and latest portfolio values.

5.2 Stateless Stream Processing

Stateless processing handles each event independently. This covers the single-event patterns from Lecture 08: filter, translator, splitter, router, and idempotent reader or writer. The first stage of the stream extension is intentionally simple because stateless processors scale and replay cleanly.

The `fx-rate-service` is the reference-data example. It polls a public FX provider on a five-minute schedule, validates the response, translates one provider response into one `FxRate` per currency pair, and publishes those records to the compacted `reference.fx.rate` topic. The compacted topic is the cache. Consumers materialize it locally and never call the FX provider on the event path, which is the Reference Data decision in Section 6.29.

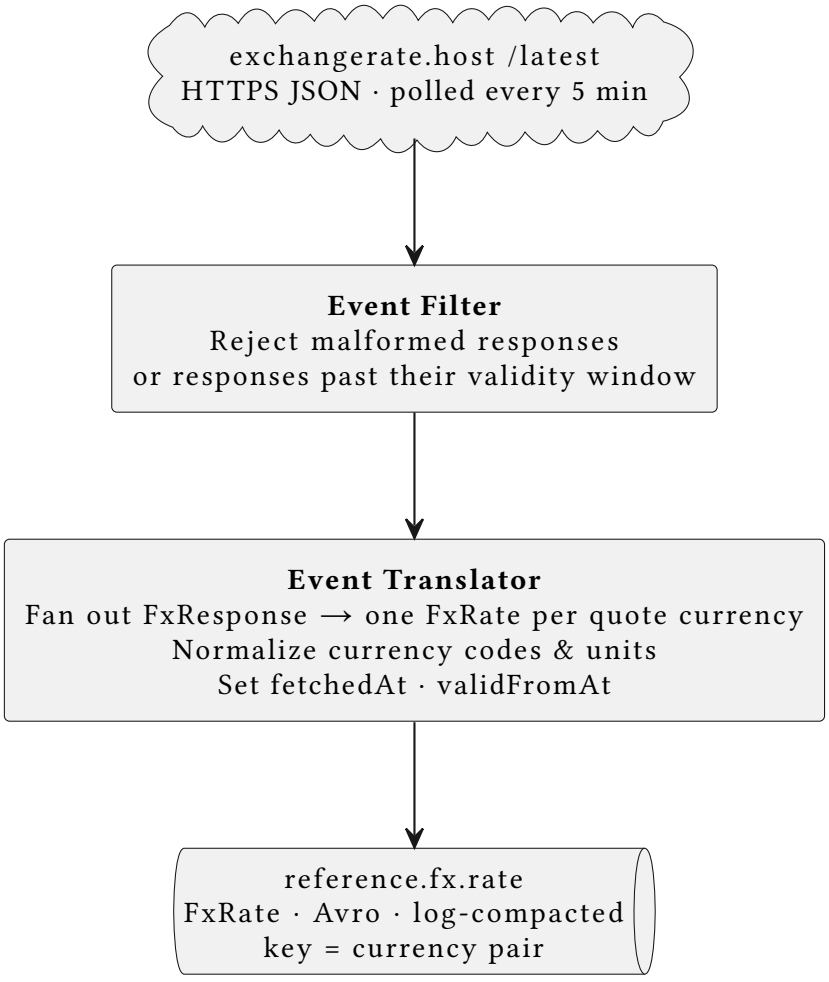


Figure 11: FX rate ingestion as single-event processing: scheduled fetch, filter, translator, compacted reference topic

The `coin-metadata-service` applies the same shape to slower-moving crypto metadata (Section 6.33). It polls CoinGecko, maps configured Binance symbols such as `BTCUSDT` to metadata such as base asset, quote asset, name, image URL, and market-cap rank, then publishes `CoinMetadata` records to `reference.crypto.metadata`. Downstream topologies can materialize this compacted topic as a `GlobalKTable`.

Cyril's ingestion path starts with `market-partial-book-ingestion-service`. It subscribes to Binance USD-M Futures Partial Book Depth streams (Section 6.21) and publishes raw depth updates as JSON to `crypto.scout.raw`. This topic is the replay boundary for Scout. Keeping the source record as raw JSON preserves observability and lets a fresh Scout application id rebuild derived scout topics from the retained raw feed (Section 6.22, Section 6.24).

The `market-order-scout-service` then performs stateless stream processing over `crypto.scout.raw`:

```
crypto.scout.raw (RawOrderBookDepthEvent, JSON)
    -> filter events without asks
    -> split ask levels into AskQuote records
    -> translate AskQuote into MatchableAsk
    -> filter AskQuote by configured ask threshold
    -> translate threshold hits into AskOpportunity
```

`AskQuote`, `AskOpportunity`, and `ScoutWindowSummary` are scout-owned Avro records (Section 6.22). `MatchableAsk` is different: Section 6.26 makes it the shared cross-service ask contract published to `crypto.scout.matchable-asks`. That keeps Transaction Service independent from Scout's dashboard-oriented event shapes.

5.3 Stateful Processing and Local State Stores

Lecture 09 adds the core stateful idea: when processing depends on previous events, records with the same key must reach the same task, and state must be persisted in local stores backed by changelog topics. CryptoFlow uses state stores for matching, windowing, portfolio valuation, and dashboard reads.

The most direct stateful topology is transaction matching. The previous trading flow matched pending orders against ticker prices kept in heap memory. Section 6.27 replaces that with a Kafka Streams topology in `transaction-service`:

```
transaction.buy-bids                crypto.scout.matchable-asks
    key: symbol                         key: symbol
    value: BuyBid                       value: MatchableAsk
        |                                   |
```

```
          └──────────── normalize symbol ─────┘
                              |
                              ▼
                  merge bid and ask candidates
                              |
                              ▼
          ValueTransformerWithKey + persistent stores
              pending-buy-bids-by-symbol
              matched-transactions
              allocated-asks
                              |
                              ▼
               transaction.order-matched
```

The topology keeps pending bids by symbol, remembers matched transaction ids, and remembers allocated ask quote ids. This makes matching restartable and prevents duplicate match decisions. A `MatchableAsk` can be allocated to at most one still-pending bid, and a transaction that has already matched is ignored on replay.

5.3.1 Place-order Matching Extension

The extension changes the matching source without turning the workflow into a stream-processing application. `placeOrder.bpmn` still creates and owns the user-visible order process, but the market decision is delegated to a Kafka Streams topology inside `transaction-service`.

When the user places an order, the place-order worker validates the user locally, persists the pending transaction, and publishes a `BuyBid` record to `transaction.buy-bids`. Independently, the market-scout pipeline converts Binance partial-book snapshots into `MatchableAsk` records on `crypto.scout.matchable-asks`. Both streams are normalized to the same symbol key before they enter the matcher.

The matcher keeps three persistent stores: pending bids grouped by symbol, transaction ids that already matched, and ask quote ids that have already been allocated. This state is what makes the extension more robust than the earlier in-memory price check. A restart can rebuild the matching state from Kafka inputs, and replay does not approve the same transaction twice or allocate one ask to multiple bids.

The business timing is event-time based. A bid is matchable for 30 seconds from its `createdAt` timestamp. The additional five seconds are not an eligibility extension; they are a retention and workflow margin aligned with the Camunda rejection timer (`PT35S`) so late processing does not immediately discard still-relevant state. A `MatchableAsk` is eligible only if its market event time falls inside the 30-second bid interval, the ask price is at or below the bid price,

and the available quantity is sufficient. If several bids compete for the same ask, deterministic price-time priority chooses the winner: highest bid price first, then earliest bid creation time, then `transactionId`.

When a match is found, the topology emits `transaction.order-matched` keyed by `transactionId`. Camunda correlates that event to the waiting process instance, after which the existing approval path continues unchanged: `approveOrderWorker` writes the approved state and outbox row, `publishOrderApprovedWorker` publishes `transaction.order.approved`, and `portfolio-service` updates holdings asynchronously. This keeps Kafka Streams responsible for market matching and Camunda responsible for the order lifecycle.

For demonstration and debugging, `transaction-service` also consumes the bid, ask, and match streams into a matching-audit projection. The audit tables are not part of the matching decision itself; they are an event-driven read model for the dashboard. They let the demo inspect recent bids, observed ask quotes, emitted matches, event-time metadata, and Kafka topic/partition/offset information without coupling the matcher to the dashboard.

The portfolio valuation topology uses state stores at a higher level of abstraction (Section 6.34). It consumes approved orders and prices, materializes both as tables, computes position values, then groups positions into a user-level aggregate:

```
transaction.order.approved
    -> holdings KTable, key = userId|symbol

crypto.price.raw
    -> prices KTable, key = symbol

holdings FK join prices
    -> position-value KTable, key = userId|symbol
    -> groupBy userId
    -> portfolio-value-store
    -> portfolio.value.updated
```

The `portfolio-value-store` backs `GET /portfolios/{userId}/streams-value`. This demonstrates the Lecture 09 interactive-query pattern: the service reads the local Kafka Streams state store for the current materialized value instead of issuing a synchronous request to another service.

5.4 Time, Windows, Grace, and Suppression

Lecture 10 distinguishes event time, ingestion time, and processing time. CryptoFlow uses event time when business meaning depends on when the market event happened, not when the processor happened to receive it.

The Market Scout summary is a simple tumbling-window aggregation. After `AskOpportunity` records are produced, the topology groups by symbol, applies a configured window size, and aggregates the opportunity count and minimum ask price into `ScoutWindowSummary`. It also materializes a dashboard stats store so the Scout dashboard can read the latest summary state.

The OHLC topology is the more complete windowing example. In the target architecture, Section 6.31 and the scope document name `crypto.price.clean` as the canonical input. The current implementation reads `crypto.price.raw` because the scope-02 price-sanity stream has not shipped yet. This is an implementation-stage deviation, not the final doctrine; the code documents the source swap as a one-line configuration change.

```
crypto.price.raw
    -> extract event timestamp from CryptoPriceUpdatedEvent
    -> groupByKey(symbol)
    -> tumbling window 1m, grace configured
    -> aggregate open/high/low/close/tickCount in ohlc-1m-store
    -> suppress until window closes
    -> join with reference.crypto.metadata GlobalKTable
    -> crypto.ohlc.1m

same source stream
    -> analogous 5m and 1h pipelines
    -> crypto.ohlc.5m / crypto.ohlc.1h
```

The grace period keeps a window open long enough to absorb bounded late ticks. `suppress(untilWindowCloses)` prevents downstream consumers from receiving every intermediate candle update. They receive one closed bar per `(symbol, window)` after the window and grace period are complete. This matches the semantics of a closed candlestick used by charts and future indicator computations (Section 6.31).

Transaction matching also uses event-time validity, but as business logic inside a stateful transformer rather than as a DSL windowed join. A bid is eligible from `BuyBid.createdAt` until `createdAt + 30s`. The five-second margin is used for state retention and the BPMN timeout alignment, not for accepting asks after the 30-second business interval. A matching ask must have `ask.eventTime` inside the 30-second interval, a price at or below the bid price, and enough quantity (Section 6.27).

5.5 Joins, Repartitioning, and Interactive Queries

CryptoFlow uses three join variants from Lectures 08 and 09.

First, the OHLC topology uses a stream-table join against `reference.crypto.metadata` (Section 6.33). It materializes metadata as a `GlobalKTable`, so each stream task can enrich closed bars locally without requiring co-partitioning between price ticks and metadata. Missing metadata does not block the bar; the left join emits the OHLC record with nullable metadata fields.

Second, the portfolio valuation topology uses a foreign-key table-table join (Section 6.34). Holdings are keyed by `userId|symbol`, while prices are keyed by `symbol`. The `Holding` value carries the symbol so Kafka Streams can join each holding row to the current price row. The result is a `PositionValue` table keyed by `userId|symbol`.

Third, portfolio valuation demonstrates multiphase repartitioning. A per-position table is the right representation for joining holdings to prices, but the dashboard needs a per-user sum. The topology therefore groups the position values by `userId` and aggregates into `portfolio-value-store`. Kafka Streams handles the repartition topic needed to move all positions for one user to the same task.

Interactive queries are used where the materialized result is directly useful to the local service API. `portfolio-service` exposes `GET /portfolios/{userId}/streams-value` from `portfolio-value-store`. `market-order-scout-service` exposes dashboard statistics from `market-scout-dashboard-stats`. The development deployment runs one instance of each service, so multi-instance host discovery and redirect logic are intentionally out of scope.

The matching audit in `transaction-service` is intentionally a regular PostgreSQL read model rather than an interactive query. It consumes the relevant Kafka topics and persists recent matching observations so the dashboard can explain why an order matched without reading directly from the matcher's internal state stores.

Pattern	Implementation	Resulting topic or store
Single-event processing	FX filter/translator, metadata polling, ask filtering and translation	`reference.fx.rate`, `reference.crypto.metadata`, `crypto.scout.ask-quotes`
Processing with local state	Transaction matching stores, OHLC window stores, portfolio stores	`pending-buy-bids-by-symbol`, `ohlc-*` stores, `portfolio-value-store`
Stream-table join	Closed OHLC bars enriched with coin metadata (Section 6.33); ADR-0030 records the planned FX/localized-price join	`crypto.ohlc.1m/5m/1h`
Table-table join	Portfolio holdings joined with latest prices by symbol	`position-value` KTable
Multiphase repartitioning	Portfolio positions re-keyed from `userId|symbol` to `userId`	`portfolio-value-store`
Windowing	Market Scout summaries, OHLC bars, bid validity interval	`crypto.scout.window-summary`, `crypto.ohlc.*`, `transaction.order-matched`
Interactive queries	Portfolio value endpoint and Market Scout dashboard	`GET /portfolios/{userId}/streams-value`, Scout dashboard state
Event-driven read model	Transaction matching audit dashboard	Bid/ask/match audit tables
Reprocessing	Fresh application id or offset reset rebuilds derived state from input topics	Rebuilt state stores and derived topics

Table 10: Implemented stream-processing patterns in CryptoFlow

5.6 Event Contracts and Reprocessing

The stream extension uses three serialization strategies because the topics have different ownership and evolution needs.

JSON for replay boundaries and legacy/raw topics `crypto.price.raw`, `crypto.scout.raw`, and existing order/user topics remain readable JSON (Section 6.3, Section 6.22). They are useful for debugging, demo inspection, and replaying source events into newer derived topologies.

Registryless Avro for scout-owned derived topics `AskQuote`, `AskOpportunity`, and `ScoutWindowSummary` are local to `market-order-scout-service` (Section 6.22). They use Avro schemas but do not need Schema Registry because no independent bounded context depends on them as stable contracts.

Schema Registry Avro for cross-service derived contracts `FxRate`, `CoinMetadata`, `Ohlc`, `PortfolioValue`, and `UserDisplayCurrencyUpdated` are registry-backed because they cross service boundaries or are consumed by independently deployable components.

`MatchableAsk` is the exception that makes the coupling boundary explicit. It is a shared Avro contract in `shared-events` because Transaction Service depends on it (Section 6.26), but it still uses the registryless generated serde in the shipped implementation. This kept the scout-to-transaction matching path aligned with the earlier Market Scout Avro setup while Schema Registry was introduced for the newer reference-data and valuation contracts.

Reprocessing follows the same rule across topologies: input topics are the source of truth for derived state. A fresh Kafka Streams `application.id`, or an offset reset for the existing app id during development, lets a topology rebuild its local stores and derived outputs from retained input records. This is useful for OHLC bars, portfolio values, and Scout summaries. The practical limit is topic retention: raw scout and market topics are intentionally short-lived in the development broker to protect disk space, while compacted reference topics retain latest state by key.

5.7 Implementation Insights

The second-half work made a clear distinction between facts, reference data, and derived views. Partial book depth and raw prices are facts from market sources. FX rates and coin metadata are reference data, represented as compacted topics. OHLC bars, ask opportunities, matched orders, and portfolio values are derived views produced by stream-processing applications.

The Market Scout split is important architecturally. `market-partial-book-ingestion-service` owns external connectivity and raw capture. `market-order-scout-service` owns the stream-processing topology and derived ask contracts. Section 6.23 keeps these responsibilities separate, and Section 6.24 gives Scout its own application id so its state and offsets can evolve independently.

The transaction matching topology moved order execution closer to the event-processing model taught in the lectures. It no longer treats ticker prices as executable liquidity and no longer loses pending orders on service restart. Persistent stores and deterministic price-time priority make the result replayable: highest bid price wins, then earliest bid creation time, then transaction id for deterministic tie-breaking.

The OHLC implementation shows how course-scope sequencing affects architecture. The intended clean-price input is documented, but the implemented topology reads `crypto.price.raw` until the price-sanity stream exists. Recording this as a stage deviation avoids turning a temporary source choice into architecture doctrine.

The portfolio valuation topology is the densest pattern demonstration (Section 6.34). It combines stream-table duality, a foreign-key table-table join, repartitioning, local state stores,

a compacted output topic, and an interactive-query endpoint. The result is still eventually consistent, but the read path is fast and local, and the state can be reconstructed from Kafka inputs.

Finally, the contract choices reflect coupling. Raw JSON keeps the platform inspectable at replay boundaries (Section 6.3). Avro without a registry is sufficient for service-local derived streams (Section 6.22). Schema Registry Avro is used where independent services need durable schema evolution (Section 6.32). This keeps the implementation pragmatic while still aligning the cross-service contracts with the lecture material on schemas and data contracts.

6 Architectural Decision Records

This chapter summarises the current architectural decision records (ADRs) referenced in the report. The authoritative ADR sources are maintained in `docs/adrs`; the summaries below condense their status, context, decision, and architectural impact.

6.1 ADR-0001: Kafka as Inter-Service Event Bus

Status: Partially superseded by ADR-0008; Kafka remains the event bus for domain events.

Context: CryptoFlow services must exchange price updates and business events without direct runtime coupling. REST would make services block on each other, while RabbitMQ would not provide Kafka's replayable log and partitioned stream model.

Decision: All inter-service domain events flow through Apache Kafka, while REST endpoints remain reserved for external clients. The platform uses a single-node KRaft broker, explicitly declared topics, manual offset handling, a dead-letter topic for poison pills, and `acks=all` for durable writes.

Consequences: Services are loosely and temporally coupled, and failed records can be isolated without stalling whole consumers. The trade-off is eventual consistency, Kafka operations, and a short retention window that prevents Kafka from acting as the long-term system of record.

6.2 ADR-0002: Event-Carried State Transfer for Price Replication

Status: Accepted

Context: `portfolio-service` needs current prices to value holdings. Calling `market-data-service` synchronously on each request would add latency and create an availability dependency.

Decision: `portfolio-service` consumes `CryptoPriceUpdatedEvent` messages and maintains an in-memory price cache using Event-Carried State Transfer. Portfolio valuation always reads from this local cache.

Consequences: Valuations remain fast and available even if market-data publishing is temporarily interrupted. The trade-off is slightly stale data during lag or reconnection windows and a short warm-up phase after cold start.

6.3 ADR-0003: JSON Serialization for Kafka Messages

Status: Accepted

Context: Kafka messages require a shared serialization format. JSON, Avro, and Protocol Buffers differ in readability, schema enforcement, and operational overhead.

Decision: CryptoFlow standardises on Jackson JSON serialization and deserialization, with event schemas defined as Java records in the `shared-events` module and type headers disabled.

Consequences: Messages are easy to inspect and no schema registry is required. The trade-off is larger payloads and the absence of broker-side schema validation or automated compatibility guarantees.

6.4 ADR-0004: Trading Symbol as Kafka Partition Key

Status: Accepted

Context: Price events must preserve per-symbol ordering so that newer prices cannot be overwritten by older ones. A poor keying strategy would distribute updates for one symbol across multiple partitions.

Decision: The Kafka message key is the trading symbol, such as `BTCUSDT`. Partition assignment uses a deterministic symbol-index mapping before falling back to hash-based routing for unknown symbols.

Consequences: All updates for one symbol stay ordered within a single partition while load remains evenly distributed for the fixed symbol set. The message key becomes a routing key only; uniqueness remains the responsibility of the event ID.

6.5 ADR-0005: Shared Library Module for Event Schemas

Status: Accepted

Context: Producers and consumers must agree on event schemas. Duplicating DTOs in every service would invite schema drift and runtime deserialization failures.

Decision: Event schemas live in a dedicated `shared-events` Maven module that is used as a compile-time dependency by every producing and consuming service. The module contains schema records and serialization dependencies only.

Consequences: Schema changes become atomic and type-safe across the codebase. The cost is build-time coupling, because services depending on `shared-events` must be rebuilt when the shared contract changes.

6.6 ADR-0006: Binance WebSocket Streams for Market Data

Status: Accepted

Context: The platform needs a live market-data source. Polling a REST API would introduce artificial delay, while paid providers would add cost and operational overhead.

Decision: `market-data-service` subscribes to Binance public WebSocket streams at `wss://stream.binance.com:9443/ws/<symbol>\@ticker` and forwards price updates into Kafka.

Consequences: The system receives sub-second push updates at zero cost and stays event-driven end-to-end. The trade-off is dependence on one upstream provider and the need to manage reconnects, heartbeats, and variable event rates.

6.7 ADR-0007: Portfolio-Service as Sole Data Owner of the Portfolio Bounded Context

Status: Accepted

Context: Portfolio holdings are stateful domain data with their own invariants. Sharing database access across services would blur ownership and make schema evolution risky.

Decision: `portfolio-service` is the exclusive owner of portfolio persistence. It uses PostgreSQL 16, manages schema changes through Flyway, keeps Hibernate in `validate` mode, and accepts cross-service interaction only through events and opaque identifiers.

Consequences: Portfolio schema evolution becomes explicit, auditable, and local to one service. The trade-off is that other services cannot join against portfolio tables and must instead rely on asynchronous integration patterns.

6.8 ADR-0008: Camunda 8 (Zeebe) as the Process Orchestration Engine

Status: Accepted; partially supersedes ADR-0001 for orchestrated workflows.

Context: CryptoFlow already implements two concrete orchestrated workflows, `userOnboarding` and `placeOrder`. Both BPMN models use Camunda 8's out-of-the-box outbound email connector for notifications, and a production-grade version of the system would need to scale many concurrent workflow instances in a cloud deployment. Pure Kafka choreography would not provide a central process-state authority or sufficient operational visibility.

Decision: CryptoFlow standardises on Camunda 8 with Zeebe for process orchestration. BPMN models are deployed from the services, workers run as stateless gRPC clients, and Kafka continues to carry domain events while Zeebe manages orchestration state and correlations. Kafka and email integration stay inside the BPMN models through Camunda connector templates rather than custom application-level client code.

Consequences: The architecture gains executable BPMN models, scalable parallel workflow execution through Zeebe's cloud-oriented design, connector-based notification steps, and strong runtime visibility through Camunda Operate. The trade-off is dependence on an external SaaS engine and a Zeebe-specific learning curve.

6.9 ADR-0009: User-Service as Owner of User Identity and Confirmation State

Status: Accepted; amended by ADR-0010.

Context: Several services depend on a verified user identity, but the system needed one service to own user lifecycle and email confirmation semantics. After ADR-0010 moved orchestration into `onboarding-service`, that ownership question still remained.

Decision: `user-service` becomes the authoritative owner of the `User` aggregate and confirmation lifecycle. It prepares and persists pending confirmation links, exposes the public confirmation endpoint, correlates `UserConfirmed` back into Camunda, publishes `user.confirmed`, and persists the user record only after confirmation.

Consequences: Identity ownership and confirmation rules are centralised in one bounded context, and every persisted user is guaranteed to be confirmed. The trade-off is additional persistence and operational surface inside `user-service`, plus the need for an externally reachable confirmation URL.

6.10 ADR-0010: Dedicated Onboarding-Service as Saga Orchestrator

Status: Accepted

Context: Registration evolved into a distributed workflow: after confirmation, both user creation and portfolio creation must complete across separate bounded contexts. Keeping that saga inside `user-service` would create unnecessary coupling.

Decision: A dedicated `onboarding-service` owns `userOnboarding.bpmn` and orchestrates the onboarding saga. It coordinates `prepareUserWorker`, waits for the user-confirmed message, fans out to `userCreationWorker` and `portfolioCreationWorker` in parallel, and uses a boundary timer to invalidate stale confirmation links.

Consequences: Orchestration is isolated from domain ownership, and the onboarding flow now models the Parallel Saga pattern explicitly. The trade-off is an additional deployable unit with its own Camunda credentials, deployment lifecycle, and monitoring needs.

6.11 ADR-0011: Event-Carried Compensation for Onboarding Failures

Status: Accepted

Context: In the onboarding saga, either local creation step can fail after the sibling service already succeeded. Compensation must therefore cross service boundaries without introducing synchronous coupling.

Decision: Compensation is carried through Kafka events. The service that detects failure deletes its own state and publishes a compensation request so the sibling service can clean up; both sides also listen defensively to the opposite compensation topic.

Consequences: Compensation remains loosely coupled and safe under at-least-once delivery because deletions are idempotent. The trade-off is a more complex failure flow and additional event choreography to understand and monitor.

6.12 ADR-0012: Flag-Driven Completion for Creation Tasks

Status: Accepted

Context: Throwing BPMN errors directly from creation workers could terminate the onboarding process before compensation branches become reachable. The saga needed a way to represent failure without short-circuiting the BPMN control flow.

Decision: `userCreationWorker` and `portfolioCreationWorker` always complete their Zeebe jobs and communicate outcomes through boolean process variables such as `isUserCreated` and `isPortfolioCreated`. Exclusive gateways then route toward success or compensation.

Consequences: Compensation paths remain reachable for every failure mode, including validation and duplicate cases. The trade-off is reduced failure visibility in Camunda Operate because technically the jobs complete successfully.

6.13 ADR-0013: Fairy Tale Saga for the `placeOrder` Workflow

Status: Accepted

Context: The `placeOrder` process contains an Event-Based Gateway that may wait for a price match for an arbitrary amount of time. Cross-service consistency with portfolio updates is required, but local ACID transactions must remain service-local.

Decision: The workflow adopts the Fairy Tale Saga pattern: synchronous Zeebe service tasks for local transactions and eventual cross-service consistency through Kafka. Deterministic business failures are handled via BPMN error boundaries rather than saga-wide compensation.

Consequences: Zeebe can suspend the workflow at the price-match wait without holding open database or network resources, and each service keeps its own transaction boundary. The trade-off is an accepted eventual-consistency window after order approval and no compensation once an order is semantically terminal.

6.14 ADR-0014: Outbox Pattern for Order Approval

Status: Accepted

Context: Approving an order requires both persisting `APPROVED` in the transaction database and publishing `OrderApprovedEvent` to Kafka. A crash between those two steps would otherwise create silent data loss.

Decision: `transaction-service` uses the Transactional Outbox pattern. The approval step writes the status change and outbox payload in one local transaction, a publication step sends the event to Kafka, and a scheduled safety net republishes stale unpublished rows after crashes.

Consequences: Approval state and event publication become atomically durable, and broker outages no longer create unrecoverable gaps. The trade-off is an extra table, a background scheduler, and the need for downstream consumers to tolerate at-least-once delivery.

6.15 ADR-0015: Portfolio Update Durability for Approved Orders

Status: Accepted

Context: After the outbox event is published, the workflow must decide whether to wait for a portfolio acknowledgement before sending the executed email. Waiting would tighten coupling between `transaction-service` and `portfolio-service`.

Decision: The executed email is sent immediately after `publishOrderApprovedWorker` completes. Portfolio propagation remains pure Event-Carried State Transfer: `portfolio-service` consumes `OrderApprovedEvent` autonomously and no acknowledgement flows back into the BPMN process.

Consequences: Approval workflows finish promptly, email latency stays deterministic, and `portfolio-service` owns its own retry and DLT logic. The trade-off is that portfolio failures are no longer visible in Camunda Operate and users may briefly see approved orders before holdings catch up.

6.16 ADR-0016: Idempotent Consumer for Portfolio Updates

Status: Accepted

Context: Kafka delivers messages at least once. If `portfolio-service` crashes after updating holdings but before committing the consumer offset, replaying the same `OrderApprovedEvent` would otherwise double the holding quantity.

Decision: `portfolio-service` inserts the `transactionId` into a `processed_transaction` table with a `UNIQUE` constraint inside the same transaction as the holding update. Duplicate inserts fail fast and cause the event to be skipped safely.

Consequences: Re-delivered approval events no longer corrupt holdings, and the processed table doubles as an audit log. The trade-off is one extra database write per update and unbounded table growth unless retention is added later.

6.17 ADR-0017: Replicated Read-Model for User Validation at Order Placement

Status: Accepted

Context: `transaction-service` must reject orders from users who have not completed registration. Synchronous validation against `user-service` would block order placement on another service's availability.

Decision: `transaction-service` maintains its own confirmed-users read model in a local database table. `user-service` publishes `UserConfirmedEvent` to a log-compacted Kafka topic, and `transaction-service` consumes the event and upserts the local projection.

Consequences: User validation becomes a local database read and remains available even if `user-service` is down. The trade-off is a small eventual-consistency window and the future need for compensating events if users can later be deactivated.

6.18 ADR-0018: Human Escalation for Deterministic Workflow Failures

Status: Accepted

Context: Some approval-flow failures are deterministic, such as a missing transaction record or a missing outbox row. Retrying these cases indefinitely would not fix the root cause and would only delay operator visibility.

Decision: Workers throw typed BPMN errors for these failures, and interrupting boundary events route the process into an operations user task in Camunda Tasklist. The task carries the relevant order context and requires a resolution note before closure.

Consequences: Deterministic integrity problems surface immediately in Tasklist with an audit trail, and the process instance stays open until acknowledged. The trade-off is additional operational monitoring and no automatic retry path from the human-escalation step.

6.19 ADR-0019: Database per Service for Stateful Bounded Contexts

Status: Accepted

Context: CryptoFlow now contains several stateful bounded contexts: user identity, portfolio management, and trading. Sharing one database across them would couple schema evolution, transactions, deployments, and failure domains.

Decision: Every stateful bounded context gets its own database: `user-service` owns `user_service_db`, `portfolio-service` owns `portfolio_service_db`, and `transaction-service` owns `transaction_service_db`. Services must not read or write each other's tables;

cross-service consistency is handled through events, replicated read models, outbox publication, and sagas.

Consequences: Structural coupling is reduced and each stateful service becomes a more independent architecture quantum. The trade-off is that cross-service joins and ACID transactions disappear, which makes eventual consistency and the supporting integration patterns a permanent architectural requirement.

6.20 ADR-0020: Transaction-Service as Sole Owner of the Trading Bounded Context

Status: Accepted

Context: Order placement, pending-order matching, approval or rejection, and reliable publication of approved trades all belong to one coherent trading domain. That domain needed a single service boundary rather than being split across portfolio, user, and onboarding concerns.

Decision: `transaction-service` is the sole owner of the trading bounded context. It owns `placeOrder.bpmn`, the Camunda workers, transaction lifecycle state, pending-order matching, publication of approved-order events, and local persistence such as `transaction_record`, `outbox_events`, and the confirmed-user validation read model.

Consequences: Trading invariants and behaviour are concentrated in one service boundary, while `portfolio-service` stays a downstream projection owner and `user-service` remains the identity owner. The trade-off is another deployable service with its own persistence model, but the responsibilities stay aligned with the domain.

6.21 ADR-0021: Binance Partial Book Depth Streams for Market Scout

Status: Accepted

Context: Market Scout needs continuous futures order-book input to derive ask-side signals and demonstrate filtering, translation, Avro contracts, and windowed processing. Binance offers request-response depth calls, partial book streams, and diff book streams.

Decision: The MVP uses Binance USD-M Futures Partial Book Depth Streams. They provide bounded top-N bid and ask levels with exchange timestamps, which fits the intended topology without requiring full local order-book reconstruction.

Consequences: Market Scout can miss ask opportunities outside the configured top-N depth, but the stream-processing topology stays simple and bounded. A future full-order-book implementation would need snapshot reconciliation, diff streams, sequence validation, and gap recovery.

6.22 ADR-0022: Avro Contracts for Derived Market Scout Events

Status: Accepted

Context: ADR-0003 standardised Kafka messages on JSON, but the Market Scout topology needs explicit contracts for translated and aggregated project-owned events while keeping raw exchange payloads replayable.

Decision: `crypto.scout.raw` remains a JSON replay boundary. Derived Market Scout events use Avro contracts for `AskQuote`, `AskOpportunity`, and `ScoutWindowSummary`, generated locally with registryless serdes.

Consequences: Raw ingestion remains inspectable and replayable, while derived topology outputs gain stronger contracts. The trade-off is a deliberate exception to the JSON default and no central schema compatibility enforcement until Schema Registry is introduced for broader cross-service derived contracts.

6.23 ADR-0023: Split Partial Book Ingestion from Market Order Scout Processing

Status: Accepted

Context: The initial Market Scout service both consumed Binance partial book streams and ran the Kafka Streams topology over `crypto.scout.raw`. That made the MVP simple, but weakened the service-boundary story around Kafka.

Decision: Split the flow into `market-partial-book-ingestion-service`, which owns Binance connectivity and raw JSON publication, and `market-order-scout-service`, which owns ask-side filtering, translation, thresholding, and windowed summaries.

Consequences: The architecture now has a clearer exchange-facing ingestion boundary and a separate domain-specific processing boundary. The trade-off is an additional Spring Boot module, Docker Compose entry, build artifact, and test surface.

6.24 ADR-0024: New Kafka Streams Application ID for Market Order Scout

Status: Accepted

Context: After ADR-0023, the Spring application name changed, which also changes the derived Kafka Streams `application.id`. Keeping the old ID would preserve offsets and state but retain coupling to the pre-split service identity.

Decision: Accept `market-order-scout-service-topology` as the new Kafka Streams application ID and do not migrate the old local state or offsets. The service may reprocess `crypto.scout.raw` from `earliest` when no committed offsets exist.

Consequences: The split gets a clean operational identity and uses the raw topic as the intended recovery path. The trade-off is possible duplicate derived records on reused local Kafka clusters, which is acceptable for the course project and local development context.

6.25 ADR-0025: Derived Market Features for Market Scout Events

Status: Accepted

Context: Partial-depth raw events already contain enough ask-side information to derive useful liquidity features, but those features belong to project-owned derived contracts rather than the exchange-facing raw contract.

Decision: Keep `crypto.scout.raw` unchanged and extend `AskQuote` and `AskOpportunity` with computed ask-side fields: `bestAskPrice`, `bestAskQuantity`, and per-level `askNotional`. `ScoutWindowSummary` remains unchanged.

Consequences: Derived scout events become more useful for ask liquidity analysis while the raw Binance capture stays stable. The trade-off is repeated best-ask context on each flattened quote and required Avro/test updates.

6.26 ADR-0026: Matchable Ask as Cross-Service Ask Contract

Status: Accepted

Context: `transaction-service` needs ask-side liquidity for matching, but should not depend on scout-local analysis events optimised for Market Scout concerns.

Decision: Introduce `MatchableAsk` in `shared-events` as the cross-service Avro contract. `market-order-scout-service` publishes it to `crypto.scout.matchable-asks`, keyed by normalised symbol, with deterministic `askQuoteId`, price, quantity, event time, and source venue.

Consequences: Trading can match against a narrow, stable contract while scout-local events remain free to evolve for analysis. The trade-off is one additional topic and Avro generation support in `shared-events`.

6.27 ADR-0027: Event-Time Bid/Ask Matching Window

Status: Accepted

Context: The previous order-placement implementation approved orders from ticker prices cached in memory, mixing valuation prices with executable liquidity and losing pending state on restart.

Decision: `transaction-service` owns a Kafka Streams topology over `transaction.buy-bids` and `crypto.scout.matchable-asks`, both keyed by symbol. A bid matches only if ask event time falls within the bid's 30-second validity window, price and quantity constraints pass, and price-time priority selects it.

Consequences: Matching now uses executable ask liquidity and event-time semantics instead of local heap state. Losing bids remain pending within their validity window, and Camunda's rejection timer is set to `PT35S` to include a bounded late/fallback margin.

6.28 ADR-0028: Display Currency as User Identity Data

Status: Accepted

Context: Users need portfolio values and buy-time quotes in a chosen currency, while user-service is the source of truth for user identity and preferences. Extending one-time confirmation events or calling user-service synchronously would blur event semantics or violate ECST.

Decision: Add Display Currency as a per-user ISO-4217 code owned by `user-service`. It is persisted with default `USD`, updated through `PATCH /users/{id}/display-currency`, and published to the compacted `user.display-currency` topic as `UserDisplayCurrencyUpdated`.

Consequences: Currency preference changes propagate through Kafka and remain display-only: holdings, orders, and snapshots stay USDT-denominated. The trade-off is a new compacted topic and the need to distinguish Display Currency from any future accounting unit of account.

6.29 ADR-0029: FX-Rate-Service as Reference Data Context

Status: Accepted

Context: Display Currency conversion needs FX rates. These are slow-moving reference data from an HTTP provider, unlike Binance WebSocket price ingestion owned by `market-data-service`.

Decision: Introduce `fx-rate-service` in a new Reference Data bounded context. It polls a public FX provider every five minutes, translates supported pairs, and publishes `FxRate` events to the compacted `reference.fx.rate` topic.

Consequences: Market data remains focused on crypto price ingestion, while slow-moving reference data has a clear owner. The trade-off is an additional deployable and a dependency on a free public provider, mitigated by compacted last-known rates.

6.30 ADR-0030: Stream-Table Join for Price Localisation

Status: Accepted

Context: USDT price streams can be converted on read or enriched stream-side. The project scope values demonstrating the stream-table join pattern in a customer-facing flow.

Decision: Record the target design for a Kafka Streams enrichment app that joins `crypto.price.clean` with the `reference.fx.rate` GlobalKTable and emits `LocalizedPrice` to `crypto.price.localized`, keyed by symbol. Each event carries the source USDT price plus a map of supported display currencies. The current implementation keeps valuation and OHLC on `crypto.price.raw` until the clean-price stream is available.

Consequences: The ADR fixes the intended localized-price contract and join shape, while the shipped system still performs implemented portfolio valuation and OHLC processing from raw USDT prices. The trade-off of the target design is another streams app, topic, and failure surface, but the topology is simple and scales with the supported currency set.

6.31 ADR-0031: Venue-Native OHLC with Read-Time Conversion

Status: Accepted

Context: OHLC bars aggregate USDT-denominated price ticks, but Display Currency raises whether bars should be emitted per currency, as a map-per-bar, or venue-native with conversion at read time. The scope also needs to demonstrate suppress-on-window-close semantics.

Decision: Emit closed OHLC bars venue-native in USDT on `crypto.ohlc.1m`, `crypto.ohlc.5m`, and `crypto.ohlc.1h`. The topology uses event-time tumbling windows and `suppress(untilWindowCloses)`, while display conversion happens at API read time using current FX rates.

Consequences: Each OHLC event represents a final closed bar, and topic count grows only with interval count. The trade-off is that historical-FX accuracy is out of scope; charts are rescaled by current FX rates rather than reconstructed with event-time FX.

6.32 ADR-0032: Avro and Confluent Schema Registry for Derived Events

Status: Accepted

Context: New derived events such as `FxRate`, `PortfolioValue`, `Ohlc`, `CoinMetadata`, and `UserDisplayCurrencyUpdated` cross multiple service ownership boundaries. `LocalizedPrice` is also defined as the ADR-0030 target contract, although the corresponding stream has not shipped yet. Registryless Avro is too informal for that level of schema evolution.

Decision: Adopt Avro with Confluent Schema Registry for those derived event types and add `schema-registry` to Docker Compose. Existing JSON topics remain JSON, and existing registryless Market Scout Avro topics are not migrated as part of this decision.

Consequences: New cross-service derived contracts gain central schema compatibility checks and generated type safety. The trade-off is one additional infrastructure dependency and build-time Avro code generation in services that use the registered contracts.

6.33 ADR-0033: Coin Metadata Enrichment via GlobalKTable

Status: Accepted

Context: OHLC consumers need display metadata such as asset names, logos, market-cap rank, base asset, and quote asset. Fetching that metadata synchronously from dashboard read paths would undermine the read-time cache pattern.

Decision: Add `coin-metadata-service` in the Reference Data context. It polls CoinGecko, publishes `CoinMetadata` to the compacted `reference.crypto.metadata` topic, and the OHLC topology joins closed bars against a `GlobalKTable` before publishing enriched `Ohlc` records.

Consequences: Dashboard consumers receive bars that already carry rendering metadata, and the OHLC scope demonstrates stream-table joins alongside windowing and suppress. The trade-off is schema growth on `Ohlc`, a static symbol-to-CoinGecko mapping, and dependence on a rate-limited public provider.

6.34 ADR-0034: Portfolio Valuation Streams App inside Portfolio-Service

Status: Accepted

Context: Portfolio valuation must continuously compute per-user value and expose it through interactive queries while demonstrating table-table join, multiphase repartitioning, and local state. The open questions were service placement, holdings source, and price source.

Decision: Host the valuation topology inside `portfolio-service` with application ID `portfolio-service-valuation`. It replays `transaction.order.approved` into a holdings KTable, uses `crypto.price.raw` as the USDT price source, keeps `at_least_once` processing, and exposes `GET /portfolios/{userId}/streams-value` beside the existing Postgres-backed endpoint.

Consequences: Scope 04 patterns are demonstrated in one user-facing flow, and rebuilding from the event log is explicit. The trade-off is that holdings exist as both a Postgres projection and a Streams state-store projection, plus new Kafka Streams and RocksDB runtime requirements in `portfolio-service`.

6.35 ADR-0035: Loop Ops Resolution Back into the `placeOrder` Happy Path

Status: Accepted; partially supersedes ADR-0018.

Context: ADR-0018 routed deterministic approval and publication failures to ops tasks that terminated the process after acknowledgement. This surfaced failures but prevented the user-facing closure email and, in the publish-failure case, left the process incomplete even though Kafka publication had already happened.

Decision: After ops completion, the process re-enters the happy path asymmetrically. Approval errors loop back to the approve step so the atomic approve-with-outbox write can retry; publish errors skip forward to the executed-email step because the Kafka event has already been published.

Consequences: Orders can still complete through the happy-path end event after human intervention, and premature ops closure self-heals by surfacing the same task again. The trade-off is a more nuanced BPMN model and a permanent audit anomaly if an outbox row was missing after Kafka publication.

7 Experiments: Results & Insights

This chapter presents the experimental results obtained during the project, covering the Kafka reliability experiments from Exercise 1 and the implementation findings they informed. The full runnable setups, command transcripts, and reproduction steps are documented in `assignments/ex-1/experiments.md`.

7.1 Kafka Producer Reliability: Message Loss with acks=0

Objective: Demonstrate that disabling acknowledgments and retries results in permanent data loss when the Kafka leader broker crashes.

Setup: A two-broker Kafka cluster (KRaft mode) with a `ClickStream-Producer` configured with `acks=0` and `retries=0`, emitting one event approximately every 150 ms. A `ClickStream-Consumer` logs received events.

Procedure: While the producer and consumer were running, the active leader was identified via `kafka-topics --describe` and then hard-killed using `docker stop`.

Result: The producer log shows three consecutive events being sent without error:

Producer log — three events sent without error (log)

```
clickEvent sent: eventID: 75, timestamp: ..., xPosition: 201,
    yPosition: 592, clickedElement: EL6,
clickEvent sent: eventID: 76, timestamp: ..., xPosition: 412,
    yPosition: 795, clickedElement: EL9,
Current Leader: 2 host: localhost port: 9092
In Sync Replicates: [2]
clickEvent sent: eventID: 77, timestamp: ..., xPosition: 606,
    yPosition: 1025, clickedElement: EL9,
```

However, the consumer log jumps directly from `eventID=75` to `eventID=77`:

Consumer log — eventID=76 missing (log)

```
Received click-events - value: {eventID=75, ...} - partition: 0
Received click-events - value: {eventID=77, ...} - partition: 0
Received click-events - value: {eventID=78, ...} - partition: 0
```

`eventID=76` was sent by the producer but never appeared in the consumer — it is permanently lost, and the producer received no error because `acks=0` means fire-and-forget.

Insight: With `acks=0`, the application has no mechanism to detect or recover from message loss: Kafka may never durably receive the record, and the producer cannot tell. Durability requires at least `acks=all` and `retries > 0`; to avoid duplicates during retry,

`enable.idempotence=true` should also be enabled. This directly supports the durable-write choice documented in Section 6.1.

7.2 Consumer Offset Behavior

7.2.1 Safe Replay with auto.offset.reset=earliest

Objective: Confirm that when no committed offsets exist, `earliest` forces a full replay of retained events.

Setup: A fresh consumer group (`grp1`) with `auto.offset.reset=earliest`. Events were produced before the consumer started.

Result: The consumer received all events from offset 0, including those produced before it joined. The consumer was then killed before auto-commit could fire (within five seconds):

```
Received click-events - value: {eventID=45, timestamp=175674327349750, xPosition=926,
Position=521, clickedElement=EL16}- partition: 0
Received click-events - value: {eventID=46, timestamp=175674494709208, xPosition=179,
Position=686, clickedElement=EL9}- partition: 0
Received click-events - value: {eventID=47, timestamp=175674656272000, xPosition=793,
Position=777, clickedElement=EL5}- partition: 0
Received click-events - value: {eventID=48, timestamp=175674814550250, xPosition=1076,
yPosition=1042, clickedElement=EL5}- partition: 0
^C%
```

Figure 12: Consumer shutdown while printing events 45–48, proving offsets were never committed

After restart, the consumer log shows a reset back to `offset=0` and replays the retained backlog from the beginning:

```
events-0
[main] INFO org.apache.kafka.clients.consumer.internals.SubscriptionState - [Consumer
lientId=consumer-grp1-1, groupId=grp1] Resetting offset for partition click-events-0
 position FetchPosition{offset=0, offsetEpoch=Optional.empty, currentLeader=LeaderAnd
och{leader=Optional[localhost:9092 (id: 2 rack: null)], epoch=0}}.
Received click-events - value: {eventID=0, timestamp=175666893516333, xPosition=581,
osition=949, clickedElement=EL2}- partition: 0
Received click-events - value: {eventID=1, timestamp=175667097478041, xPosition=1835,
Position=246, clickedElement=EL3}- partition: 0
Received click-events - value: {eventID=2, timestamp=175667259613791, xPosition=182,
osition=826, clickedElement=EL18}- partition: 0
Received click-events - value: {eventID=3, timestamp=175667431334583, xPosition=1488,
```

Figure 13: Consumer restart replaying from `eventID=0` after offset reset

Insight: If no committed offsets exist, `earliest` replays the oldest records Kafka still retains, so replay is deterministic for the retained backlog. This is useful for at-least-once processing and short-term recovery, but it does not preserve the topic's full history forever: once the retention policy deletes older records, they cannot be replayed. The operational cost is that restarts may require processing the full retained backlog again.

7.2.2 Data Gap with auto.offset.reset=latest

Objective: Show that `latest` causes new consumer groups to miss all previously retained events.

Setup: A fresh consumer group (`grp2`) with `auto.offset.reset=latest`. Retained events existed on the topic.

Result: The client joins `grp2` with no offsets and immediately resets to the tail of the topic:

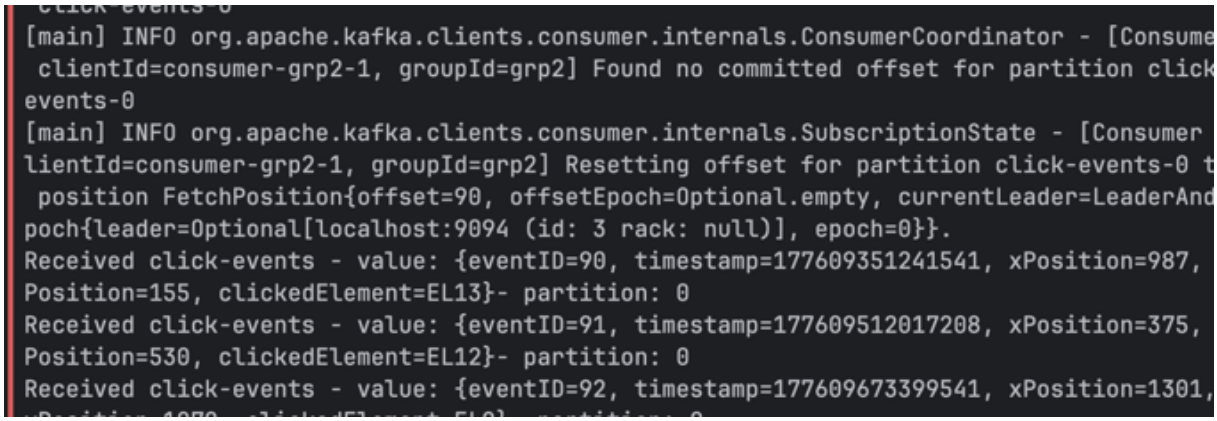


Figure 14: Consumer with `latest` resetting directly to offset 90, skipping all retained history

The producer emitted IDs `0..6` before the consumer printed anything; those remain unread by `grp2`:

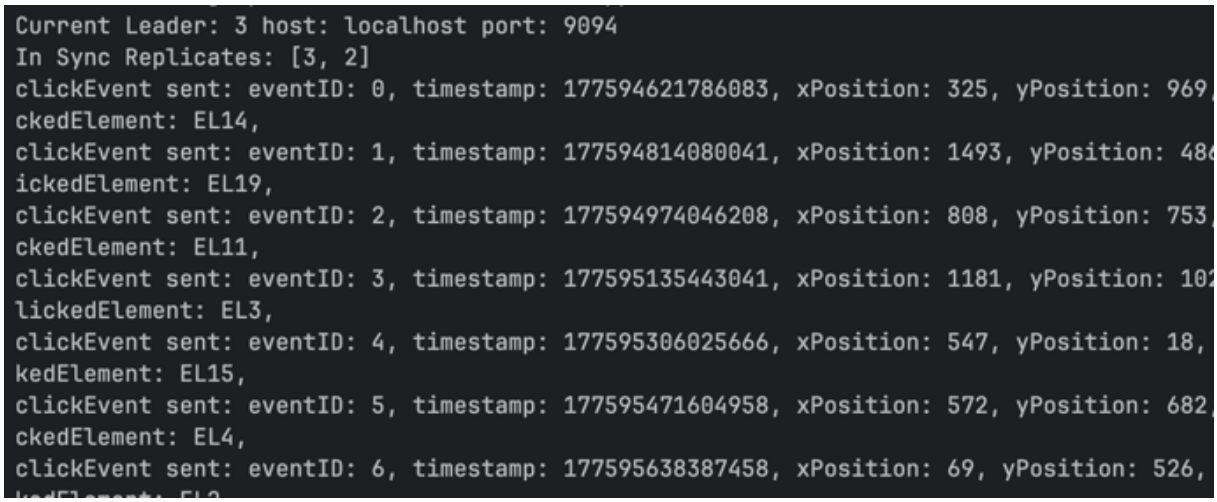


Figure 15: Consumer only receiving newly produced events, historical data effectively lost

Insight: `latest` is unsafe for workloads that require replay of retained data. Explicit use of `earliest`, manual `seek`, or pre-seeded offsets is necessary to prevent silent data gaps.

7.3 Fault Tolerance: Leader Failover Timing

Objective: Measure the availability gap during a leader broker crash and verify that no acknowledged data is lost.

Setup: A three-broker Kafka cluster with `acks=all`. The active leader was identified via `kafka-topics --describe` and hard-killed using `docker compose kill -s KILL kafkaX` during active production.

Result: The producer log shows the last acknowledged event before the crash and the recovery sequence:

Last acknowledged event before crash — log

```
ACKED id=212 partition=0 offset=212    (18:12:51.570)
```

The leader changed from broker `2` to broker `3`, detected at `18:13:00.375`. The measured timings were:

Metric	Value
Last ACK to leader elected	8805 ms
Last ACK to first recovered ACK	8890 ms
First ACK after recovery	85 ms

Table 11: Leader failover timing measurements

The first event acknowledged after recovery was `id=213`, and the consumer processed events `211..271` with no gaps.

Insight: With `acks=all`, leader failover was an availability event, not a durability event. No acknowledged data was lost. The more useful outage measurement is from the last successful ACK to the first recovered ACK, not from the leader-election log line, because most of the pause elapsed before the election was reported.

7.4 Fault Tolerance: Data Loss with acks=1

Objective: Demonstrate that leader-only acknowledgment (`acks=1`) can lose records when the leader crashes before replication completes.

Setup: The same three-broker cluster, but with `acks=1` and `retries=0`. Replication lag was artificially increased via `replica.fetch.wait.max.ms=3000` to widen the loss window.

Result: The producer log around the crash shows the last acknowledged events and the queued events that were not yet replicated:

Logs around leader crash — log

```
1 ACKED id=70 partition=0 offset=70
2 ACKED id=71 partition=0 offset=71
3 CLICK_EVENT_QUEUED id=72 ...
4 CLICK_EVENT_QUEUED id=73 ...
```

After recovery, the consumer log reveals the gap:

Consumer log after recovery — gap at eventID=71 log

```
RECEIVED eventID=70 partition=0 offset=70 ...
GAP DETECTED from=71 to=71 previous=70 current=72
RECEIVED eventID=72 partition=0 offset=71 ...
RECEIVED eventID=73 partition=0 offset=72 ...
```

`eventID=71` was acknowledged by the leader but never reached the followers before the crash — permanently lost. Note that `eventID=72` now appears at `offset=71` on the new leader, confirming the record was never replicated.

Insight: `acks=1` only guarantees that the leader appended the record; it says nothing about replication to followers. The `replica.fetch.wait.max.ms=3000` setting deliberately widened the loss window enough to trigger manually; on a real localhost cluster replication is sub-millisecond. For durable writes, the safe baseline is `acks=all`, plus retries and producer idempotence.

For the complete fault-tolerance experiment logs, timings, and configuration notes, see `assignments/ex-1/experiments.md`.

7.5 Implementation Observations

7.5.1 Event-Carried State Transfer Performance

The `LocalPriceCache` in the portfolio service, implemented as a `ConcurrentHashMap`, handles the six-symbol price feed without contention issues. With Binance pushing updates at variable rates (sub-second during active markets), the cache stays current within milliseconds under normal conditions. The 503 warm-up behavior — returning an error until the first price event arrives — was validated as an effective guard against serving stale-from-startup values.

7.5.2 Portfolio Update Idempotency After Consumer Failure

During integration work, we accidentally reproduced the failure mode later documented in Section 6.16. Taking `portfolio-service` offline after processing an `OrderApprovedEvent` but before the Kafka offset was safely committed caused the same event to be delivered again on restart. Without a deduplication guard, a `2 BTC` order was applied twice and the portfolio incorrectly showed `4 BTC`.

The fix is the idempotent-consumer design from Section 6.16: before applying the holding update, `portfolio-service` inserts the event's `transactionId` into `processed_transaction` under a `UNIQUE` constraint inside the same transaction as `upsertHolding()`. If the insert fails, the event is a replay and is skipped; if it succeeds, both the deduplication marker and the holding update commit atomically. This turned an accidental failure into concrete validation of the need for consumer-side idempotency under at-least-once delivery.

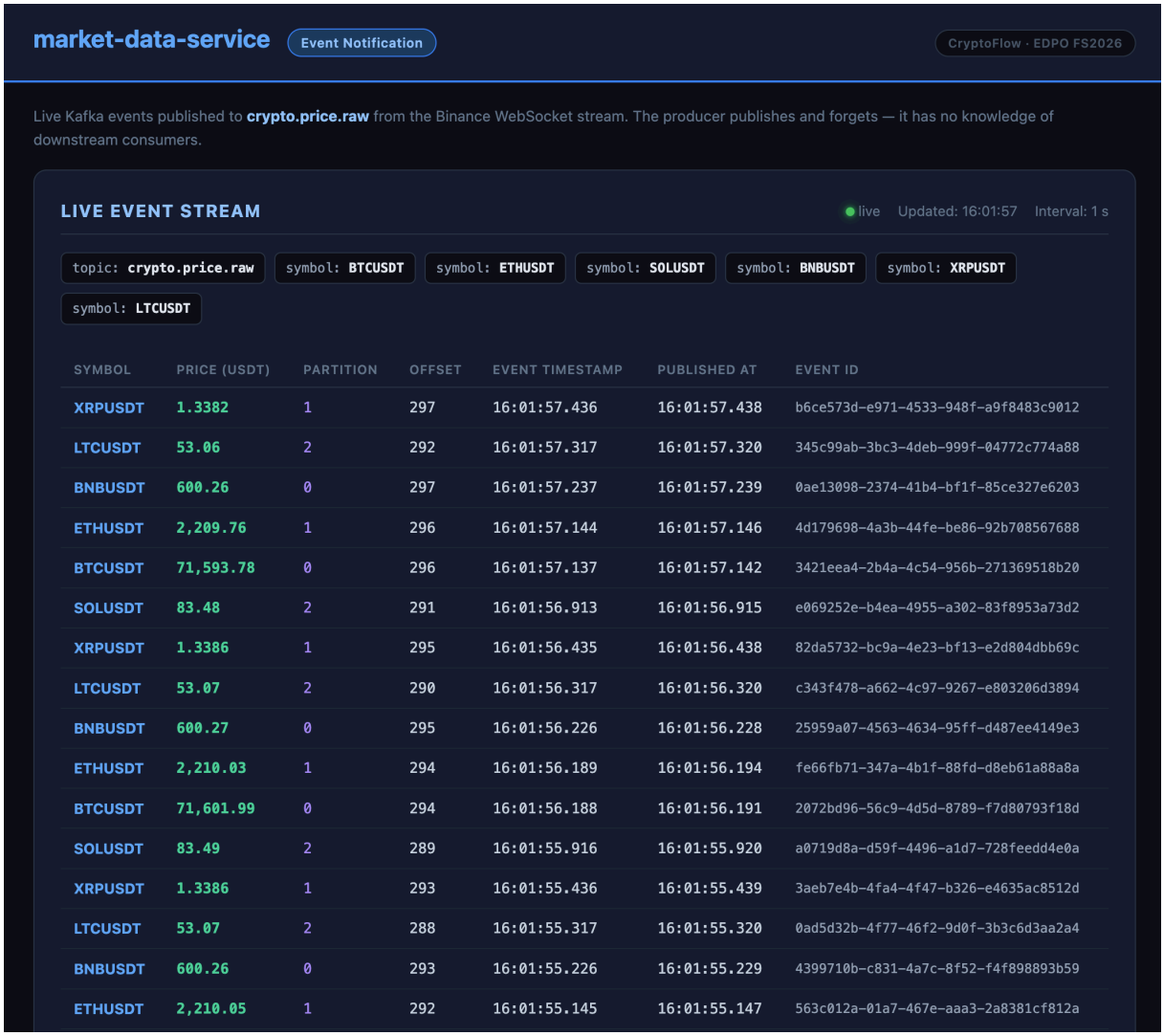
7.5.3 Saga Compensation Reliability

The onboarding saga's compensation flow was tested by deliberately triggering failures in the user and portfolio creation workers. The flag-driven completion pattern (ADR-0012) proved effective: the BPMN process consistently reached the correct compensation branch regardless of which creation step failed. The dual Kafka listener pattern (both services listening for both compensation topics) provides an additional safety net, though no cases were observed where the primary compensation path failed.

8 Live System Demonstration

CryptoFlow ships purpose-built dashboards for the user-facing and process-facing services. Every page polls its own backend on a one- to five-second interval and renders the current system state without any manual refresh. The figure slots below document the demo surfaces; the visible red notes mark the screenshot replacements delegated before hand-in.

8.1 market-data-service — Event Notification



SYMBOL	PRICE (USDT)	PARTITION	OFFSET	EVENT TIMESTAMP	PUBLISHED AT	EVENT ID
XRPUSDT	1.3382	1	297	16:01:57.436	16:01:57.438	b6ce573d-e971-4533-948f-a9f8483c9012
LTCUSDT	53.06	2	292	16:01:57.317	16:01:57.320	345c99ab-3bc3-4deb-999f-04772c774a88
BNBUSDT	600.26	0	297	16:01:57.237	16:01:57.239	0ae13098-2374-41b4-bf1f-85ce327e6203
ETHUSDT	2,209.76	1	296	16:01:57.144	16:01:57.146	4d179698-4a3b-44fe-be86-92b708567688
BTCUSDT	71,593.78	0	296	16:01:57.137	16:01:57.142	3421eea4-2b4a-4c54-956b-271369518b20
SOLUSDT	83.48	2	291	16:01:56.913	16:01:56.915	e069252e-b4ea-4955-a302-83f8953a73d2
XRPUSDT	1.3386	1	295	16:01:56.435	16:01:56.438	82da5732-bc9a-4e23-bf13-e2d804dbb69c
LTCUSDT	53.07	2	290	16:01:56.317	16:01:56.320	c343f478-a662-4c97-9267-e803206d3894
BNBUSDT	600.27	0	295	16:01:56.226	16:01:56.228	25959a07-4563-4634-95ff-d487ee4149e3
ETHUSDT	2,210.03	1	294	16:01:56.189	16:01:56.194	fe66fb71-347a-4b1f-88fd-d8eb61a88a8a
BTCUSDT	71,601.99	0	294	16:01:56.188	16:01:56.191	2072bd96-56c9-4d5d-8789-f7d80793f18d
SOLUSDT	83.49	2	289	16:01:55.916	16:01:55.920	a0719d8a-d59f-4496-a1d7-728feedd4e0a
XRPUSDT	1.3386	1	293	16:01:55.436	16:01:55.439	3aeb7e4b-4fa4-4f47-b326-e4635ac8512d
LTCUSDT	53.07	2	288	16:01:55.317	16:01:55.320	0ad5d32b-4f77-46f2-9d0f-3b3c6d3aa2a4
BNBUSDT	600.26	0	293	16:01:55.226	16:01:55.229	4399710b-c831-4a7c-8f52-f4f898893b59
ETHUSDT	2,210.05	1	292	16:01:55.145	16:01:55.147	563c012a-01a7-467e-aaa3-2a8381cf812a

Figure 16: market-data-service dashboard showing the live Kafka event stream

TODO: Update this screenshot to show the current dashboard with the OHLC candlestick chart, coin metadata panel, interval selector, chart statistics, and live event stream.

The market-data-service dashboard (Figure 16) combines the raw event notification stream with the implemented OHLC topology. The upper panel renders a candlestick chart from closed OHLC bars that the co-located Kafka Streams application writes to `crypto.ohlc.1m`, `crypto.ohlc.5m`, and `crypto.ohlc.1h`. The symbol selector, interval selector, bar count, last close, and percentage-change statistics are read from the dashboard's local OHLC snapshot.

When the metadata join has produced enriched bars, the coin panel shows the asset name, base/quote pair, logo, and market-cap rank from `reference.crypto.metadata`.

The lower panel still exposes the raw `crypto.price.raw` feed. Each row corresponds to one Kafka record and shows the symbol, USDT price, partition, offset, original exchange timestamp, producer publication time, and event UUID. This makes the Event Notification pattern visible while also showing how the same source stream is reused by downstream topologies without changing the producer.

8.2 portfolio-service — Event-Carried State Transfer

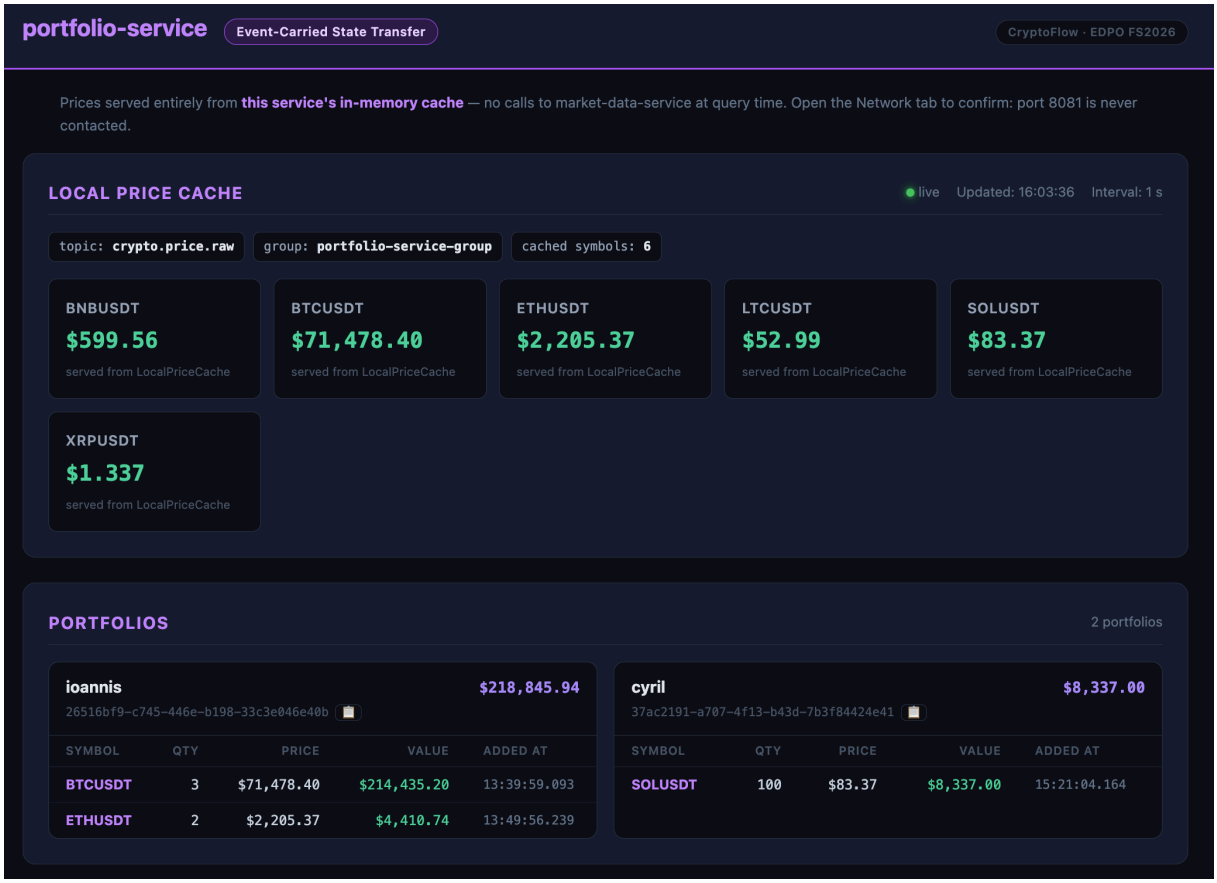


Figure 17: portfolio-service dashboard showing the local price cache and all portfolio holdings

TODO: Update this screenshot to show the current dashboard with USDT price cache, FX cache, portfolio display-currency selector, converted totals, and copied user IDs.

The portfolio-service dashboard (Figure 17) shows three local read-side replicas. The USDT price cache is filled from `crypto.price.raw`, the FX cache is filled from `reference.fx.rate`, and the display-currency cache is filled from `user.display-currency`. Because these caches are populated by consuming events rather than by querying other services at request time, no dashboard request calls market-data-service, fx-rate-service, or user-service synchronously.

This is the defining property of Event-Carried State Transfer: the consumer owns the local state it needs.

The portfolio cards show the user's name, UUID, copied user-id shortcut, display-currency selector, converted total, USDT subtotal when applicable, current FX rate, and a holdings table with per-symbol quantity, display-currency price, converted value, and creation timestamp. The running implementation performs conversion at API read time from the USDT price cache and FX cache. The ADR-0030 `crypto.price.localized` stream remains a target design, not the path used by this dashboard. The small propagation delay between an approved order in transaction-service and a new holding in portfolio-service illustrates the latency introduced by the asynchronous event flow.

8.3 transaction-service — Bid/Ask Matching, Outbox, and Replicated Read-Model

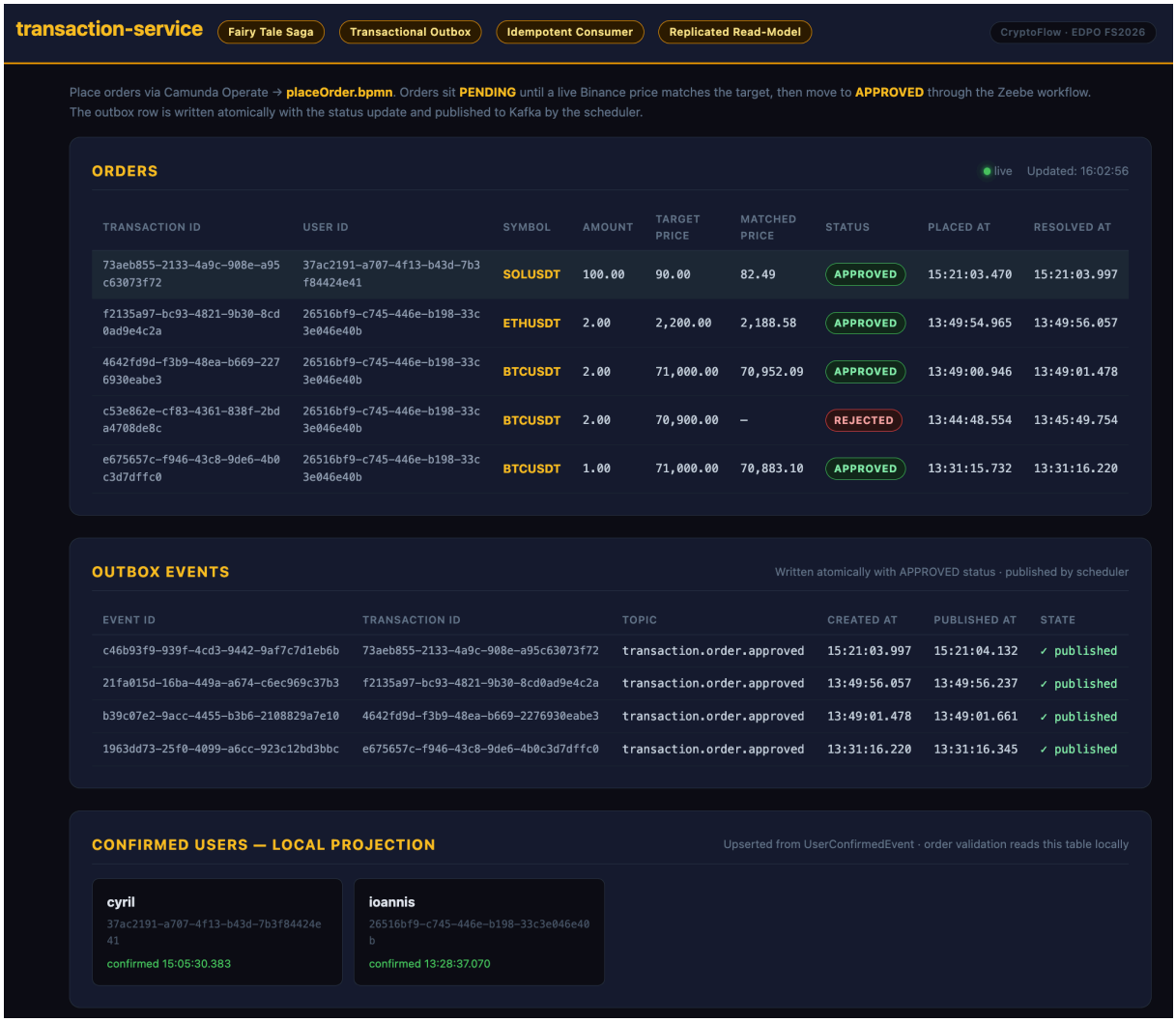


Figure 18: transaction-service dashboard showing orders, outbox events, and the local confirmed-user projection

TODO: Update this screenshot to show the current dashboard with buy-time quote widget, orders, outbox events, confirmed-user projection, and matching audit.

The transaction-service dashboard (Figure 18) starts with a buy-time quote widget. It reads the user's display currency from the local `user.display-currency` cache, the latest USDT price from the local `crypto.price.raw` cache, and the current FX rate from `reference.fx.rate`. The widget previews the display-currency price that the user sees before placing an order through Camunda; it is a read-side helper, while actual order placement still goes through `placeOrder.bpmn`.

The Orders table lists all placed orders with their status (`PENDING`, `APPROVED`, or `REJECTED`), the target price specified by the user, the matched ask price that triggered the transition, and the timestamps for placement and resolution. Orders remain `PENDING` until the transaction matching topology allocates a `MatchableAsk` to the order's `BuyBid` and emits `transaction.order-matched`. Only then does the Zeebe workflow advance and the `approveOrderWorker` write the `APPROVED` status.

The Outbox Events table confirms the Transactional Outbox pattern: each approved order has a corresponding row in the `outbox_event` table that was written atomically in the same database transaction as the status update. A scheduler running every 500 ms reads unpublished rows, publishes them to Kafka, and marks them as published. The `Published At` timestamp and the checkmark in the State column prove that the event has left the service's local store. So long as the database transaction commits, the event will eventually be published. This eliminates the dual-write problem.

The Confirmed Users section shows the service's local `confirmed_user` projection, which is populated by consuming `UserConfirmedEvent` messages from Kafka. It also displays the cached display currency so a confirmed user can be selected directly for the quote widget. Order validation reads this projection to check whether a requesting user has completed onboarding, without making a synchronous call to the onboarding-service or user-service. This is the Replicated Read-Model pattern: each service keeps a local, eventually-consistent copy of the data it depends on, trading some storage for autonomy and resilience.

The Matching Audit section ties the stream-processing extension back to the workflow. It joins the recently observed `BuyBid`, `MatchableAsk`, and `transaction.order-matched` audit rows in the dashboard response so each match can be inspected with bid price and quantity, ask quote id, matched price and quantity, event-time timeline, source venue, and Kafka topic/partition/offset metadata.

8.4 market-order-scout-service — Windowed Ask-Price Distribution

TODO: Add screenshot of the market-order-scout-service dashboard.

Figure 19: market-order-scout-service dashboard showing windowed minimum ask-price distributions

The market-order-scout-service dashboard (Figure 19) consumes the materialized dashboard state exposed by its Kafka Streams topology. It shows one panel per symbol with a histogram of `log(minAskPrice)` across the retained window summaries, the number of retained windows, and the statistical moments mean, variance, skewness, and kurtosis. A separate topics section lists the raw input topic, derived ask-quote stream, matchable-ask stream, ask-opportunity stream, and window-summary topic used by the service.

8.5 onboarding-service — Parallel Saga via Camunda

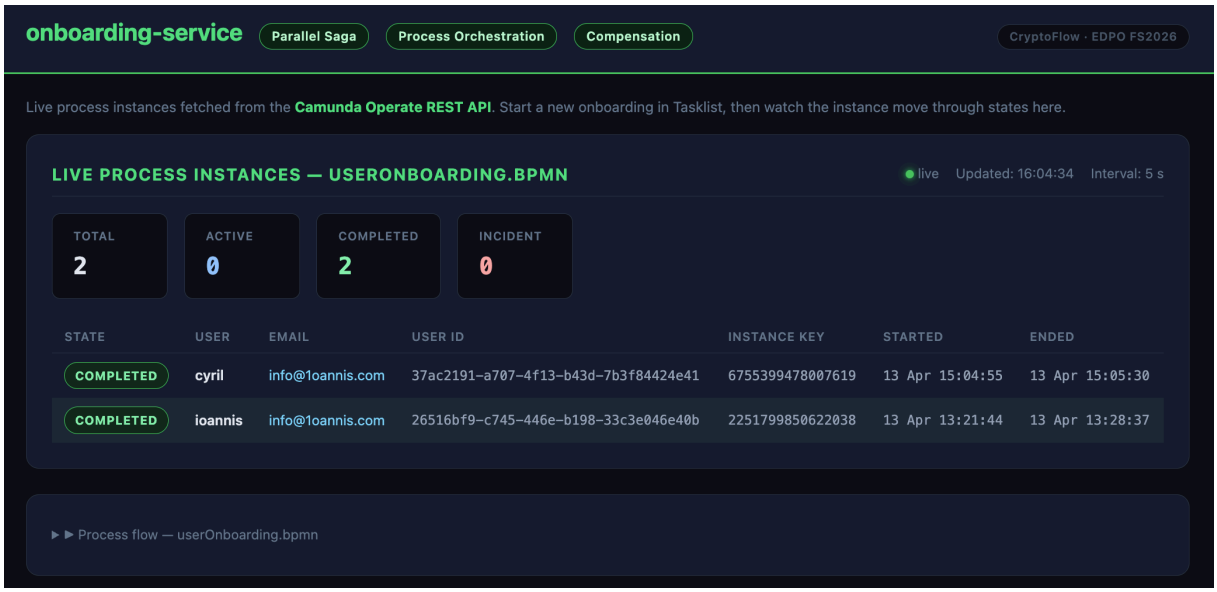


Figure 20: onboarding-service dashboard showing live Camunda process instances enriched with user variables

TODO: Update this screenshot to show the current dashboard with process instance statistics, the instance table, error banner state, and expandable process-flow section.

The onboarding-service dashboard (Figure 20) queries the Camunda Operate REST API and displays the twenty most recent instances of the `UserOnboarding` BPMN process. The page summarizes process-instance statistics, lists instances in a table, and exposes an expandable process-flow section for the onboarding BPMN. Each row shows the instance state, the user name, email address, generated user UUID resolved from Zeebe process variables, the full instance key, and the start and end timestamps. Active instances have a pulsing green state badge, while completed instances are rendered in a muted style.

The dashboard is filtered to the latest deployed version of the process definition, so re-deployments during development do not mix old and new instances in the same view. Enriching the instance list with user-level variables requires Operate variable queries for each displayed instance; the results are joined in the onboarding-service response before the browser renders them. An error banner appears if the Operate API is unreachable, making connectivity issues immediately visible without breaking the rest of the page.

9 Reflections & Lessons Learned

9.1 Technical Lessons

9.1.1 Process-oriented Architecture

Camunda BPMN has a steep learning curve, but the payoff is real. Learning how to model gateways, timers, compensation, and message correlations in Camunda was initially demanding. Once this hurdle was overcome, however, the modeler supported development very well because it made the flow explicit and reduced the implementation effort for individual tasks. A good example was the confirmation email, where Camunda's mail connector replaced what would otherwise have required a custom SMTP client in application code.

The concepts are intuitive in theory, but implementation multiplies the decisions. Patterns such as orchestration, compensation, and event-driven communication feel straightforward when discussed in lectures or on paper. The complexity appeared during implementation, where many additional degrees of freedom had to be resolved: event structure, correlation keys, error handling, retries, and service boundaries all influenced each other. Precise decision-making turned out to be essential, because without it, it is easy to get lost in technically plausible but incompatible alternatives.

Dual-write problems are easy to overlook. The need for the transactional outbox (ADR-0014) only became obvious once we considered what happens when the application crashes between a database commit and a Kafka publish. Before that, the inline publish felt natural and correct. The lesson is that dual-write hazards hide behind code that looks straightforward — they only surface under failure conditions that are rare but not impossible.

Idempotency must be designed in, not patched on. The duplicate portfolio update we observed during integration (documented in Chapter 7) confirmed that at-least-once delivery is not a theoretical edge case. It happens during normal restarts and rebalancing. Retrofitting idempotency after the fact required rethinking the transaction boundary around `upsertHolding()`. Designing consumers as idempotent from the start would have been simpler than fixing the bug after discovery.

9.1.2 Stream-processing Architecture

Stream-processing topologies become complex quickly once state enters the design. Stateless filters and translators were easy to reason about, but bid/ask matching, OHLC windows, and portfolio valuation introduced state stores, repartitioning, joins, grace periods, suppression, and reprocessing questions. The main lesson was to draw the topology and

its keys before writing code; otherwise small contract decisions later force larger topology changes.

Compacted topics are useful caches, not just transport channels. The FX-rate, coin-metadata, user-display-currency, and portfolio-value topics showed how Kafka can hold the latest value per key and let services rebuild local state without synchronous lookups. This pattern made the read paths simpler, but it also forced us to think carefully about topic cleanup policy, keys, and whether each event represented a durable latest fact.

Event time is a correctness boundary, not only metadata. OHLC aggregation, market-scout summaries, and bid/ask matching all became easier to reason about once processing was driven from payload timestamps instead of arrival time. Custom timestamp extractors made replay deterministic and made late-event behavior explicit, but they also forced every event contract to define which timestamp actually carries business meaning.

Kafka Streams DSL and Processor API solve different kinds of problems. The DSL was concise for filters, translators, table joins, windows, and aggregates. The bid/ask matcher showed its limit: a normal stream-stream join would produce candidate pairs, but it could not express allocation, price-time priority, and "each ask only once" semantics cleanly. Dropping to the Processor API was the right trade-off because it kept the streaming-join semantics while making the state and ordering rules explicit.

Interactive Queries couple read paths to runtime placement. The portfolio valuation endpoint demonstrated how powerful it is to read directly from a materialized state store. At the same time, it made the single-instance assumption visible: a production deployment would need metadata lookup and request forwarding for keys hosted on another instance. Interactive Queries are therefore a useful pattern, but they should be introduced with an explicit scaling story.

Single-broker Kafka obscures real durability semantics. Running the experiments on multi-broker clusters and then developing against a single-broker Docker Compose stack created a subtle gap: `acks=all` behaved identically to `acks=1` in development, so producer reliability issues were invisible locally. We only recognized this fully when documenting the configuration for the report. For future projects, even a two-broker local cluster would surface replication behavior earlier.

9.2 Process Lessons

9.2.1 Process-oriented Architecture

ADRs were even more important here than in ASSE. Because implementation decisions were tightly coupled across services, documenting them close to the point of decision was

essential. A choice made in one service often constrained or informed the design of another, so the rationale had to stay visible and reusable. Kafka event contracts are the simplest example: once we settled on how events should be structured and exchanged, that decision had to be applied consistently across multiple services.

The lecture concepts transferred well to a self-chosen project. One of the most valuable aspects of this project was applying the lecture concepts to a topic we chose ourselves. This freedom made the work more engaging, while also forcing us to implement the patterns under realistic constraints instead of only discussing them abstractly.

Choosing the right saga pattern requires understanding the wait semantics. The distinction between the Parallel Saga for onboarding and the Fairy Tale Saga for order placement was not immediately obvious. It only became clear once we analyzed the nature of the wait: onboarding waits for a deterministic human action (clicking a link), while order placement waits for a non-deterministic market event (a price match). The wait semantics dictated the coupling model, which in turn dictated the saga style. This selection process was one of the most instructive parts of the project.

9.2.2 Stream-processing Architecture

Stream-processing work needed a stronger up-front vocabulary. During the second half of the course, we had to become stricter about names such as raw order-book depth, ask quote, matchable ask, opportunity, localized price, OHLC bar, and portfolio value. Without this vocabulary, it was too easy to call every market-data event a "price stream" and blur important ownership boundaries. The context documents and ADRs helped keep stream contracts precise.

The unit of planning shifted from services to topologies. In the process-oriented half, most discussions started with BPMN processes, workers, and bounded contexts. In the stream-processing half, useful planning started with input topics, keys, timestamps, state stores, repartition steps, and output topics. This changed the way we estimated work: a small feature could imply several contracts, internal changelog topics, test fixtures, and replay scenarios.

Raw and derived streams should be separated deliberately. Splitting Binance partial-book ingestion from the Market Order Scout topology added one deployable, but it made `crypto.scout.raw` a replayable boundary and kept derived stream logic out of the WebSocket adapter. The same principle applied to the portfolio valuation stream: the source topic remained the durable fact, while the state store and compacted output were rebuildable projections.

Workflow timers and stream windows must be designed together. The order-matching flow crossed both halves of the course: Camunda waited for a match, while Kafka Streams evaluated bids and asks within an event-time window. Aligning the 30 s validity window, 5 s processing and retention margin, and `PT35S` BPMN rejection timer made the boundary deterministic. Designing these values separately would have created race conditions between late stream events and the workflow timeout path.

9.3 Teamwork Lessons

Clear service ownership reduces coordination overhead. Assigning each bounded context to a single team member, with shared responsibility for architecture and integration, worked well for a two-person team. Each person could develop their services independently, meeting only to align on event schemas and BPMN process contracts. The `shared-events` module served as the explicit interface between areas of ownership.

Integration issues surface late. Despite clear contracts, the majority of bugs appeared during integration: message deserialization mismatches, incorrect Zeebe correlation keys, and compensation events missing required fields. Running the full Docker Compose stack earlier and more frequently, rather than testing services in isolation, would have caught these issues sooner.

Derived-event splitting creates more data and more contracts than expected. Splitting raw order-book snapshots into ask quotes, matchable asks, opportunities, and summaries made the architecture cleaner, but it also multiplied schemas, topics, and retention choices. The split was still worthwhile because each topic had a clear consumer and lifecycle, but it made the operational surface larger than a single direct stream would have been.

Two people, many deployables. The scope was ambitious for a two-person team within one semester. The decision to add dedicated services for onboarding, reference data, and market-scout processing, while architecturally sound, added deployment and monitoring overhead. In retrospect, these trade-offs were worthwhile for demonstrating the patterns cleanly, but we underestimated the time required for BPMN process debugging, stream processing, and compensation testing.

Declaration of Authorship

«I hereby declare,

- that I have written this thesis independently,
- that I have written the thesis using only the aids specified in the index;
- that all parts of the thesis produced with the help of aids have been declared;
- that I have handled both input and output responsibly when using AI. I confirm that I have therefore only read in public data or data released with consent and that I have checked, declared and comprehensibly referenced all results and/or other forms of AI assistance in the required form and that I am aware that I am responsible if incorrect content, violations of data protection law, copyright law or scientific misconduct (e.g. plagiarism) have also occurred unintentionally;
- that I have mentioned all sources used and cited them correctly according to established academic citation rules;
- that I have acquired all immaterial rights to any materials I may have used, such as images or graphics, or that these materials were created by me;
- that the topic, the thesis or parts of it have not already been the object of any work or examination of another course, unless this has been expressly agreed with the faculty member in advance and is stated as such in the thesis;
- that I am aware of the legal provisions regarding the publication and dissemination of parts or the entire thesis and that I comply with them accordingly;
- that I am aware that my thesis can be electronically checked for plagiarism and for third-party authorship of human or technical origin and that I hereby grant the University of St. Gallen the copyright according to the Examination Regulations as far as it is necessary for the administrative actions;
- that I am aware that the University will prosecute a violation of this Declaration of Authorship and that disciplinary as well as criminal consequences may result, which may lead to expulsion from the University or to the withdrawal of my title.»

By submitting this thesis, I confirm through my conclusive action that I am submitting the Declaration of Authorship, that I have read and understood it, and that it is true.

Declaration of Authorship

St. Gallen, 22/05/2026

Ioannis Theodosiadis - 25-603-457
Cyril Gabriele - 21-558-358

Directory of writing aids

In preparing this report, we utilized the following writing aids to enhance the quality of our work:

Table 12 summarizes the tools used and the parts of the project they supported.

Aid	Usage / Application	Affected Areas
OpenAI - ChatGPT 5.4	• Grammar and Style Correction • Clarity and Coherence	All chapters
Anthropic - Claude Opus 4.6, Claude Sonnet 4.6	• Code Generation	All modules
OpenAI - Codex 5.3, Codex 5.4	• Code Generation	All modules

Table 12: Directory of writing aids